MRI PROJECT · GUIDE 1 OF 2

Understanding MMedPO from the Ground Up

A self-contained guide to *"MMedPO: Aligning Medical Vision-Language Models with Clinical-Aware Multimodal Preference Optimization"* (Zhu, Xia et al., ICML 2025). It is written for a reader who has missed the last five years of AI and has little medical background.



Prepared 29 October 2026 for the MRI project
Companion: Guide 2, *Running MMedPO on the HGX cluster*
Paper: arXiv 2412.06141 · code: github.com/aiming-lab/MMedPO

Contents

How to use this guide 3

PART I · JUST ENOUGH MEDICINE

Medical images: what the model looks at 6
Reports, questions, and why mistakes matter 10

PART II · THE AI YOU MISSED

Machine learning in a nutshell 14
Language models and the transformer 17
Teaching a language model to see 22
Hallucination: when the model stops looking 26

PART III · LEARNING FROM PREFERENCES

Learning from comparisons: RLHF and DPO 30
LoRA: fine-tuning a giant cheaply 35

PART IV · MMEDPO ITSELF

The problem MMedPO attacks 38
Step 1: building the preference data 41
Steps 2 and 3: scoring and weighted training 45

PART V · THE EVIDENCE

How the paper measures success 50
What the results show 53

PART VI · GOING FURTHER

Reading the paper and its neighbours 59
Glossary 64

How to use this guide

This guide has one job: by the time you reach the end, you should be able to open the MMedPO paper and read it comfortably, equations included. You should then be able to pick up the papers it builds on without feeling lost.

The paper itself is only nine pages. It is short because it assumes a lot: that you know what a "vision-language model" is, why it "hallucinates", what "DPO" and "LoRA" mean, what a chest X-ray report looks like, and how BLEU scores work. Each of those is a few years of progress in AI or a few weeks of medical training. This guide fills in exactly the pieces the paper needs, in the order you need them, and nothing more.

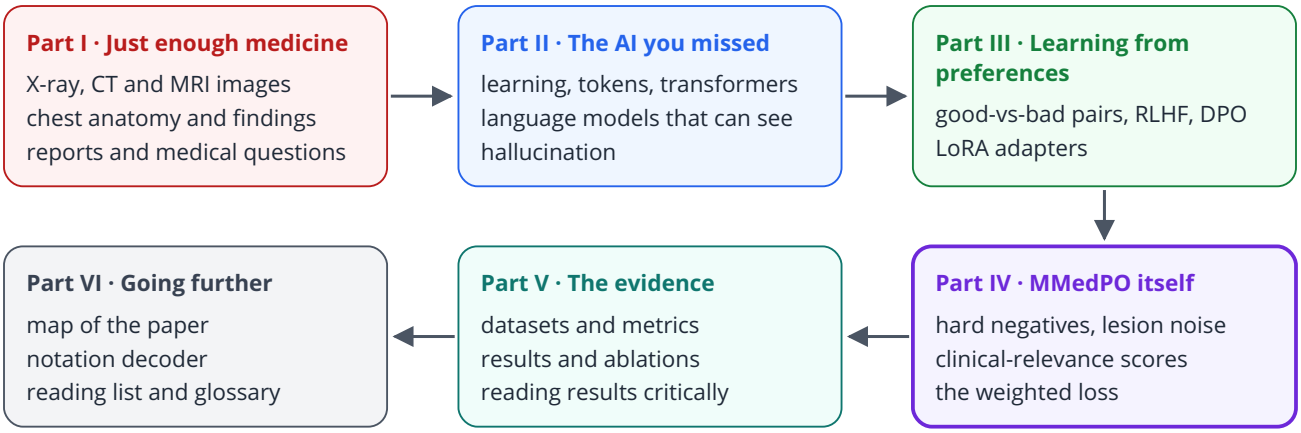


Figure 1. The route through this guide. Each part uses only ideas from the parts before it. If you already know a topic, skim its chapter recap box and move on.

The boxes you will meet

KEY IDEA
The one thing to remember from a section.

INTUITION
An everyday analogy for a technical idea.

IN THE PAPER
Where the idea shows up in MMedPO, with section and equation numbers.

MEDICAL NOTE
A piece of clinical background.

WATCH OUT
A common confusion, a notation trap, or a weak spot in the paper.

CHAPTER RECAP
A summary at the end of each chapter.

Colours carry meaning in every diagram. Blue is a model or AI component, teal is image data, orange is text, green marks the *preferred* (good) answer, red marks the *dispreferred* (bad) answer, and purple marks what MMedPO adds.

Real data. The X-ray, CT and MRI images, reports and model answers in this guide come from the test sets we downloaded to the cluster and from the MMedPO checkpoints we ran there (Guide 2 explains how). Where a number or picture is made up to explain an idea, the caption says *illustrative*.

Maths. You need no maths beyond fractions, logarithms and the idea of a probability. Each equation from the paper is unpacked one symbol at a time, with a worked example.

ABOUT THE PROJECT NAME
Your project folder is called “MRI”, but MMedPO is not specifically about MRI scans. It covers medical images in general: mostly chest X-rays, plus the CT and MRI scans that appear in two of its question-answering datasets. Chapter 1 explains the differences.

Part I

Just enough medicine

What medical images look like, what doctors write about them, and why an AI that invents findings is dangerous. Two short chapters, no prior knowledge assumed.

CHAPTER 1

Medical images: what the model looks at

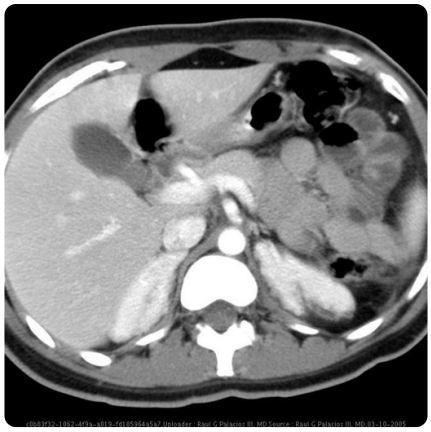
MMedPO trains an AI that looks at a medical image and either answers a question about it or writes a report. Before we talk about the AI, we need to know what these images are and how a doctor reads them.

Three ways to see inside the body

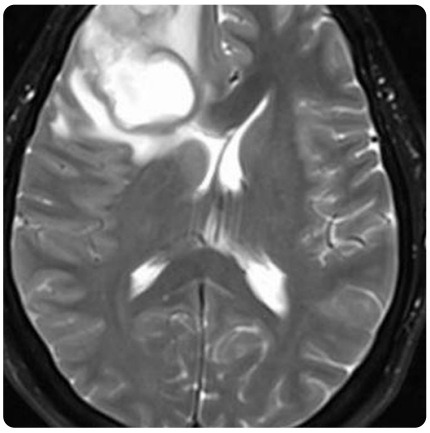
The datasets in the paper use three kinds of image. Each one makes a picture of the inside of the body in a different way, so each one shows different things well.



X-ray (radiograph)
One flat shadow picture. Fast and cheap. Most of MMedPO's data are chest X-rays.



CT (computed tomography)
Many X-rays from all angles, combined by a computer into thin slices. This one is a slice through the abdomen.



MRI (magnetic resonance)
Uses a strong magnet and radio waves, not X-rays. Excellent for soft tissue such as the brain (shown).

Figure 2. Three images from the VQA-RAD test set on our cluster (images 0001, 0004 and 0028). A CT or MRI "slice" is like looking at one slice of a loaf of bread from above; an X-ray is like a shadow of the whole loaf.

Why X-rays are white, grey and black

An X-ray machine shoots X-rays through the body onto a detector. Dense material such as bone or metal blocks most of them, so the detector receives little and that area shows up **white**. Air lets almost everything through, so air-filled lungs show up **dark**. Soft tissue, blood and fluid block a medium amount and appear **grey**. Radiologists (doctors who specialise in reading images) call a whiter-than-expected area an *opacity* and a darker-than-expected area a *lucency*.

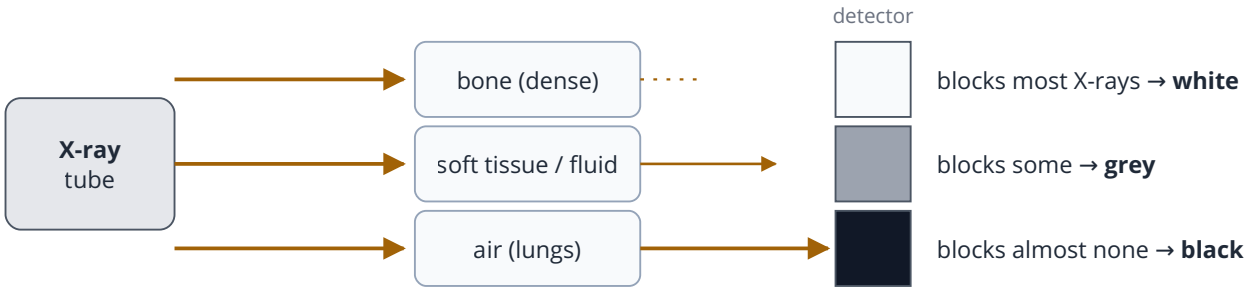


Figure 3. How an X-ray picture forms. The more a tissue blocks the beam, the whiter it looks. A lung filled with fluid (for example in pneumonia) turns from dark to white, which is how doctors spot it.

Reading a chest X-ray

Almost all the report-writing data in MMedPO are **chest X-rays**, usually taken from the back to the front (called a PA, or postero-anterior, view). Here is one from the IU-Xray test set with the structures a report talks about.

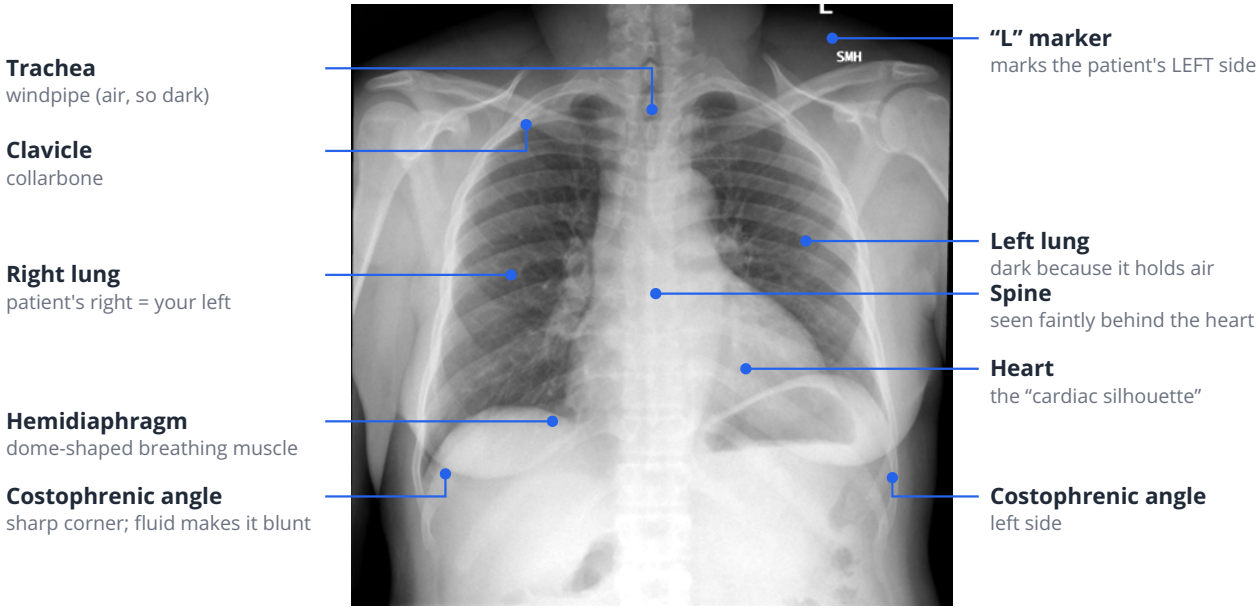


Figure 4. A frontal chest X-ray from the IU-Xray test set (study CXR3993) with the main landmarks. Images are shown as if the patient faces you, so the patient's left is on your right; the "L" letter is placed by the technician to avoid confusion. This patient's report says the heart is *mildly enlarged*; we reuse this image in Chapter 10.

> **MEDICAL NOTE: THE "CARDIOMEDIASTINAL SILHOUETTE"**
>
> The **mediastinum** is the central column of the chest between the two lungs: heart, big blood vessels, windpipe and gullet. On an X-ray they merge into one white shape, called the *cardio–mediastinal silhouette*. The **hilum** (plural hila) is where vessels and airways enter each lung. Reports describe their size and outline ("contours").

What can go wrong: the findings vocabulary

A **finding** is anything a doctor notices on the image, normal or not. The table lists the findings you will see most often in chest X-ray reports, including the ones that come up in MMedPO's examples.

Term	Meaning in plain words	What it looks like on the X-ray
Cardiomegaly	Enlarged heart	Heart wider than about half the chest width
Pleural effusion	Fluid between the lung and the chest wall	White area at the bottom of a lung; blunted costophrenic angle
Pneumothorax	Air leaking outside the lung, which collapses it	Dark area with no lung markings; visible lung edge
Consolidation / airspace disease	Air sacs filled with fluid or pus, often pneumonia	Fluffy white patch in a lung
Atelectasis	Part of the lung partly collapsed (not fully inflated)	White band or patch, often at the bases
Edema	Fluid in the lung tissue, often from heart failure	Hazy white lungs, lines near the edges
Nodule / mass	Round spot; a mass is a large one (over 3 cm)	Round white blob
Fracture	Broken bone, for example a rib	Break or step in the bone outline
Osseous / bony	Relating to bones	“No acute osseous abnormality” = no new bone problem

Table 1. Common chest X-ray findings. “Acute” means new or sudden; “chronic” means long-standing; “unremarkable” means normal.

The word “lesion”

MMedPO talks a lot about **lesions** and **lesion regions**. A lesion is any area of abnormal tissue: a tumour, an infection, a bleed, a fluid collection. On an image, the *lesion region* is the part of the picture that shows the abnormality. A doctor's eye goes straight to it. One of the paper's two key ideas (Chapter 10) is to force the AI to pay attention to exactly that region.

> CHAPTER RECAP
>
> - X-ray, CT and MRI are three ways to image the body. Most MMedPO data are chest X-rays.
> - On an X-ray, dense means white and air means black. Disease shows up as unexpected white or dark areas.
> - Reports describe the heart, mediastinum, lungs, pleura (lung lining) and bones, using a small vocabulary of findings.
> - A *lesion region* is the part of the image showing the abnormality. MMedPO cares about it explicitly.

CHAPTER 2

Reports, questions, and why mistakes matter

MMedPO is tested on two jobs: answering questions about an image, and writing a full radiology report. This chapter shows what each job looks like with real examples, and why a wrong answer here is worse than a wrong answer in most other AI applications.

The radiology report

After reading an image, the radiologist writes a **report**. It usually has a **Findings** section (what is seen, region by region) and an **Impression** (the conclusion, often one line). Here is a real report from the IU-Xray test set, the one for the X-ray in the anatomy figure of Chapter 1.

```
The heart is mildly enlarged. Left hemidiaphragm is elevated. There is no acute
infiltrate or pleural effusion. The mediastinum is unremarkable.
```

Translated: *the heart is a bit bigger than normal; the left half of the diaphragm sits higher than usual; there is no new lung infection and no fluid around the lungs; the central chest looks normal.* Notice two things. First, most of the sentences are **negative findings** ("no X, no Y"). Normal structures far outnumber abnormal ones, so reports are full of these. Second, the abnormal sentence ("mildly enlarged") is the one that matters clinically, and it is easy to drown among the normal ones.

> **WATCH OUT: "XXXX" IN IU-XRAY**
> Many IU-Xray reports contain the string `XXXX`, as in "The XXXX are intact." These are words removed to protect patient privacy (often dates or words that looked like names). They are not an error in our files.

Task 1: medical visual question answering (Med-VQA)

In **visual question answering** the model gets an image plus a question and must answer it. Med-VQA datasets split questions into two kinds:

- **Closed-ended** questions have a fixed set of answers, usually yes or no: "Is there airspace consolidation on the left side?" Scored by *accuracy* (right or wrong).
- **Open-ended** questions need a free answer: "What bright white structures form an X?" Scored by *recall*, roughly "how many of the words in the correct answer appear in the model's answer" (Chapter 12).



QUESTION (CLOSED)

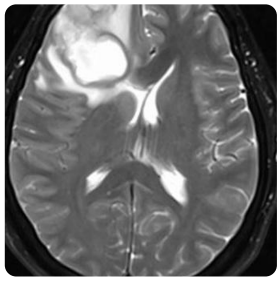
is there airspace consolidation on the left side?

CORRECT ANSWER

yes

MMEDPO MODEL ON OUR CLUSTER

Yes ✓



QUESTION (OPEN)

what bright white structures form an x?

CORRECT ANSWER

lateral ventricles

MMEDPO MODEL ON OUR CLUSTER

"The bright white structures that form an x are the corpus callosum and the anter…" ✗

Figure 5. Two real VQA-RAD test questions with the answers produced by the released MMedPO checkpoint, run on our cluster. In the second, the model names a real brain structure, but the wrong one. The lateral ventricles are fluid-filled spaces that look bright on this type of MRI. This kind of fluent, plausible, wrong answer is what the paper calls a hallucination.

Task 2: radiology report generation

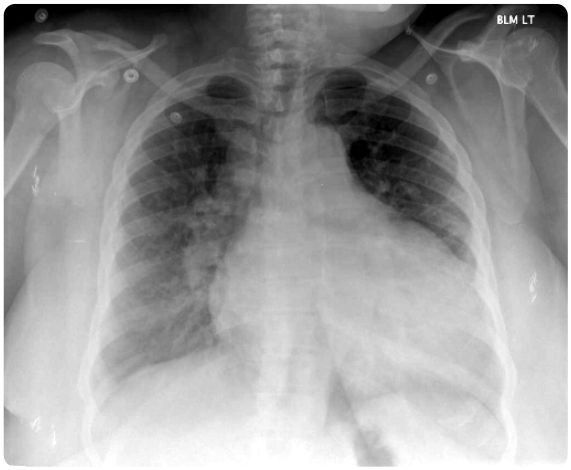
Here the model sees only the image and an instruction such as *"Walk through the important details of the image"*, and must write the whole report. It is harder: there is no question to focus it, the answer is long, and every sentence can be right or wrong. The paper tests this on two chest X-ray datasets, IU-Xray and MIMIC-CXR.

Why mistakes here are expensive

An AI that writes a slightly wrong movie review costs nothing. In medicine, two kinds of error matter:

- **A missed finding** (false negative): the report says "normal" when there is disease. The patient may not get treatment.
- **An invented finding** (false positive): the report mentions a problem that is not there. This leads to anxiety, extra tests, or wrong treatment.

Here is a real example of the first kind from our own cluster run. The image is from the IU-Xray test set; the model is the released MMedPO checkpoint for IU-Xray.



WHAT THE RADIOLOGIST WROTE

There is **moderate cardiomegaly**. There are **bilateral interstitial opacities**, increased since the previous exam. No focal airspace consolidation, pleural effusions or pneumothorax. No acute bony abnormalities.

WHAT THE MODEL WROTE

The chest X-ray shows a **normal heart size** and mediastinal and hilar contours. **The lungs are clear**, and there is no evidence of pleural effusion or pneumothorax. The impression is that there is no acute cardiopulmonary process.

Figure 6. IU-Xray study CXR1965 with the true report (top) and the report from the MMedPO IU-Xray checkpoint (bottom). The model wrote a fluent "normal" report and missed both abnormal findings. Even after MMedPO, models still make this error. The paper reduces it; it does not remove it.

Why would a model write "normal heart size" for an obviously big heart? Because most chest X-rays in its training data *are* normal, so "normal heart size … lungs are clear … no acute cardiopulmonary process" is the most likely-sounding report. The model has learned what reports usually say rather than what this image shows. Chapter 6 gives this problem its name, and the rest of the guide is about how MMedPO fights it.

CHAPTER RECAP

- Reports have Findings and an Impression. Most sentences are negative findings; the few abnormal ones matter most.
- Med-VQA has closed (yes/no, scored by accuracy) and open (free text, scored by recall) questions. Report generation writes a whole report.
- Medical AI errors are missed findings or invented findings. Both are harmful.
- Models tend to write the "typical" normal report even when the image is abnormal. Keep this example in mind.

Part II

The AI you missed

From “a model is a function with knobs” to large language models that can look at pictures, and why they sometimes describe things that are not there. This is the background the paper assumes in its first two pages.

CHAPTER 3

Machine learning in a nutshell

Every idea later in this guide (fine-tuning, DPO, LoRA, the MMedPO loss) is a variation on one recipe: a function with adjustable knobs, a score for how wrong it is, and a rule for turning the knobs. This chapter is that recipe.

A model is a function with knobs

A **model** is a mathematical function: it takes an input and produces an output. What makes it a machine-learning model is that its behaviour is controlled by a large number of adjustable numbers called **parameters** (also called *weights*). In formulas they are written as one symbol, θ (Greek "theta"), which stands for all of them together.

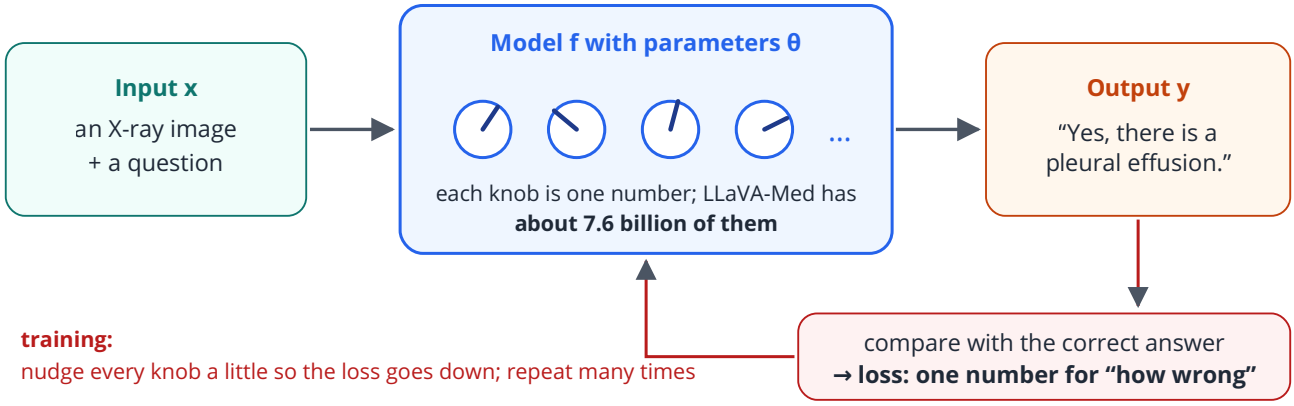


Figure 7. The machine-learning recipe. During **training**, the model's answers are compared with correct answers and its knobs are adjusted (red loop). During **inference**, which is what we ran on the cluster, the knobs stay fixed and the model just produces outputs (black arrows only).

Nobody sets the 7.6 billion knobs by hand. They start out random, and the training procedure tunes them automatically using examples.

Learning: turning knobs to reduce the loss

To train a model we need a **loss function** (or simply *loss*): a formula that takes the model's output and the correct answer and returns a single number, larger when the model is more wrong. Training means finding knob settings that make the average loss over many examples small.

The standard method is **gradient descent**. For each knob, calculus tells us which direction (turn up or turn down) would reduce the loss and how strongly. This direction is the *gradient*, and the algorithm that computes it for billions of knobs at once is called *backpropagation*. We then move every knob a small step in its downhill direction, and repeat.

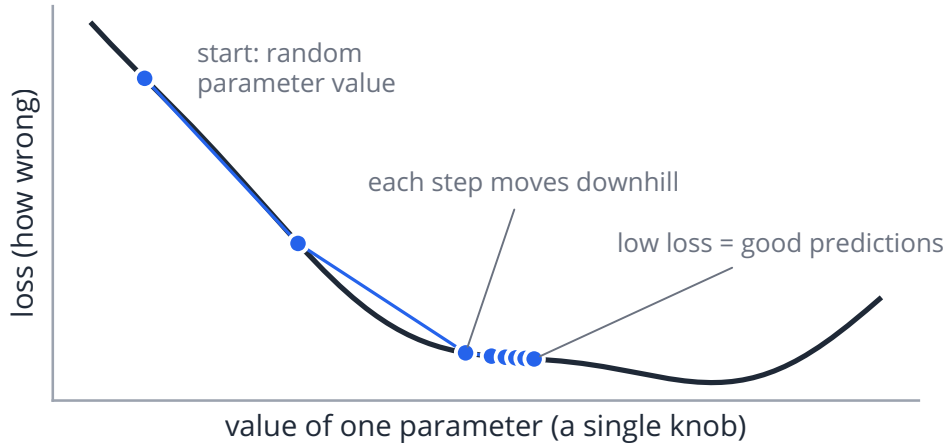


Figure 8. Gradient descent in one dimension (illustrative). Imagine the loss as a hilly landscape and the knob value as your position: you take small steps downhill. Real models do this in billions of dimensions at once.

Three words from training recipes appear in the paper, so let us pin them down:

- **Learning rate**: the size of each step. MMedPO uses 1×10^{-7} (0.0000001), which is very small. It nudges an already-trained model gently rather than rebuilding it.
- **Batch size**: how many examples are averaged before each step. MMedPO uses 4.
- **Epoch**: one full pass over the training data. MMedPO trains for 3 epochs.

> **INTUITION**
> Tuning an old radio by ear: you turn the dial a little, listen to whether the static gets better or worse, and keep turning in the direction that helps. Gradient descent does this for billions of dials at once, and calculus "hears" the direction for each dial without trial and error.

Neural networks and what "7B" means

The functions used today are **neural networks**: many *layers* stacked one after another. Each layer takes a list of numbers, multiplies it by a big table of parameters (a *matrix*), adds a few more parameters, and applies a simple bend (a *non-linearity*) so the stack can represent complicated patterns. Stack enough layers and the network can map pixels to diagnoses or words to the next word.

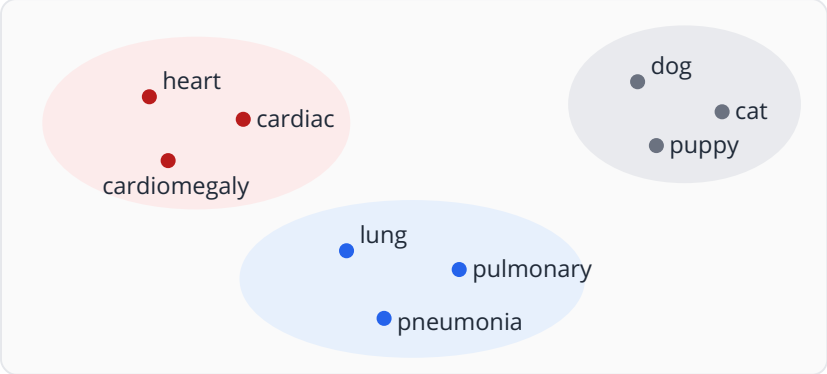
Models are named by their parameter count. "Mistral-7B" has about 7.2 billion parameters, hence "7B". Each parameter is stored as a number taking 2 bytes in the usual half-precision format (called fp16 or bf16), so the model alone needs about $7.2 \times 2 \approx 14.5$ GB of memory. This is why the model has to run on a GPU with lots of memory (Guide 2 measures it: about 16 GB including the image part).

Training data and test data

A model that memorises its training examples is useless on new patients. So every dataset is split. The model learns from the **training set** and is evaluated on a separate **test set** it never saw. All the scores in the paper, and all the timings in Guide 2, are on test sets. When the paper says "IU-Xray has 590 test reports", those 590 were held out from training.

Everything becomes numbers: embeddings

Neural networks only process numbers, so words and images must be turned into lists of numbers called **vectors**. A word's vector is its **embedding**. Embeddings are learned during training, and they end up placing words with similar meanings close together, as if every word had coordinates on a map.



Each word becomes a point.

Words used in similar ways end up near each other.

Here each point has 2 numbers so we can draw it. In Mistral-7B each token has **4,096** numbers, and image patches get the same treatment (Chapter 5).

Figure 9. An embedding space (illustrative, squashed to 2 dimensions). Distance on this map roughly means difference in meaning. This is how a network can "know" that cardiomegaly is related to the heart.

CHAPTER RECAP

- A model is a function controlled by parameters θ. Training adjusts them to reduce a loss; inference just uses them.
- Gradient descent takes small downhill steps; learning rate, batch size and epochs describe how.
- "7B" = 7 billion parameters $\approx$ 14 GB at 2 bytes each.
- Words (and later image patches) become vectors called embeddings; similar meanings sit close together.

CHAPTER 4

Language models and the transformer

The engine inside MMedPO's model is a large language model (LLM). This chapter explains what an LLM actually computes: probabilities for the next word piece. That one idea is the key to the equations in the paper.

Text is split into tokens

A language model does not read letters or whole words. It reads **tokens**: common words and word pieces from a fixed list called the *vocabulary*. Mistral-7B's vocabulary has 32,000 tokens, and each has an ID number. Here is what the real tokenizer on our cluster does to two report sentences (the _ symbol marks the start of a new word):

"The heart is mildly enlarged." → 9 tokens

`_The` `_heart` `_is` `_mild` `ly` `_en` `larg` `ed` `.`

IDs: 415, 3031, 349, 16583, 346, 481, 20832, 286, 28723

"There is moderate cardiomegaly." → 9 tokens

`_There` `_is` `_moderate` `_card` `i` `ome` `gal` `y` `.`

Everyday words are single tokens; rare medical words get chopped into pieces ("cardiomegaly" becomes five). From now on, think of any text, question or answer as a sequence of tokens, written $y = (y_1, y_2, \ldots, y_T)$.

The one job of a language model: predict the next token

A language model reads a sequence of tokens and outputs a **probability for every token in the vocabulary**: how likely it is to come next. That is all it does.

Text so far:

```
"…Findings: The heart size is ___"
```

The model assigns a probability to all 32,000 tokens. The top few are shown on the right; the remaining ones share the last 5%.

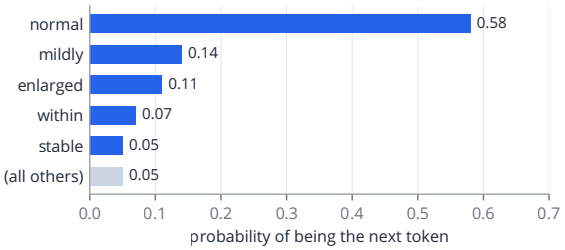
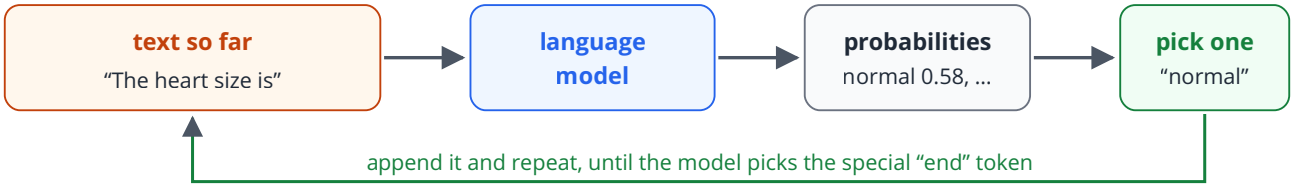


Figure 10. Next-token prediction (illustrative numbers). Probabilities over the whole vocabulary add up to 1. Note that "normal" wins because most reports in the training data say so, whatever the image shows. Keep this in mind for Chapter 6.

To **generate** a full answer, the model repeats this step: pick a token from the distribution, append it to the text, and predict again. This is called **autoregressive** generation ("auto" = self, "regressive" = feeding back), and it is what the paper means by "autoregressively predicts the probability distribution of the next token" in Section 2.1.



"The heart size is" → "... normal" → "... normal." → "... normal. The" → ... → [end]

Figure 11. Autoregressive generation. A 64-token report takes 64 trips around this loop. That is why, in Guide 2, report generation is slower than yes/no questions.

How do we "pick one"? Always taking the most likely token is called *greedy* decoding. *Sampling* picks randomly according to the probabilities, and a setting called **temperature** sharpens (low temperature) or flattens (high temperature) the distribution first. The MMedPO inference scripts use temperature 0.2, which is close to greedy. The paper's data-making step does the opposite: it samples the model "multiple rounds" at a higher temperature to collect several different answers (Chapter 10).

The probability of a whole answer, and why we use logarithms

This section is the most important one in the chapter for reading the paper's equations.

Given an input x (for us: image plus question), the model gives a probability to any complete answer y. It is the product of the next-token probabilities along the way. The paper writes this as

$$\pi_\theta(y \mid x) = p(y_1 \mid x) \times p(y_2 \mid x, y_1) \times \ldots \times p(y_T \mid x, y_1, \ldots, y_{T-1})$$

Read $\pi_\theta(y \mid x)$ as "the probability that the model with parameters θ produces answer y when given input x." The vertical bar means "given". The letter π ("pi") is borrowed from reinforcement learning, where a model that chooses actions is called a **policy**. In this paper, π just means "the model".

Multiplying many numbers below 1 quickly gives tiny numbers, so everyone works with the **logarithm** instead. The log of a product is the sum of the logs, which is easier to handle:

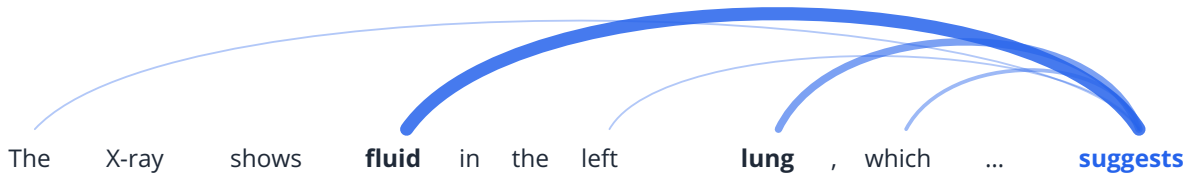
Token	probability	log probability
The	0.90	−0.105
heart	0.80	−0.223
is	0.95	−0.051
enlarged	0.30	−1.204
.	0.90	−0.105
Whole answer	**0.185** (product)	**−1.689** (sum)

Table 2. Probability of a five-token answer (illustrative numbers). Log probabilities are always ≤ 0; closer to 0 means more likely. "Raising the log probability of an answer" means making the model more likely to say it.

> **KEY IDEA**
>
> Every training method in this guide, including MMedPO, works by pushing $\log \pi_\theta(\text{answer} \mid \text{input})$ up for answers we like and down for answers we don't. The equations differ only in *which* answers, *how hard*, and *compared to what*.

Attention and the transformer

How does the model compute those probabilities? Since 2017 the answer has been an architecture called the **transformer**. Its central trick is **attention**: when processing each token, the model looks back at all earlier tokens and decides how much each one matters for the current one.



Line thickness = how much attention "suggests" pays to each earlier word (illustrative).
It looks hardest at "fluid" and "lung", which helps it predict the next word: "effusion".

Figure 12. Attention lets each token gather information from the tokens that matter for it. In a vision-language model, the earlier tokens include the *image* tokens, so attention is also how the model "looks" at parts of the picture. The paper's Figure 5 visualises exactly this for image tokens.

A transformer is a stack of identical **layers**. Each layer has an attention step (mix information between tokens) and a feed-forward step (process each token's vector on its own). Mistral-7B has 32 such layers. Each token is represented by a vector of 4,096 numbers that gets refined layer by layer. After the last layer, the vector at the final position is converted into the 32,000 next-token probabilities.

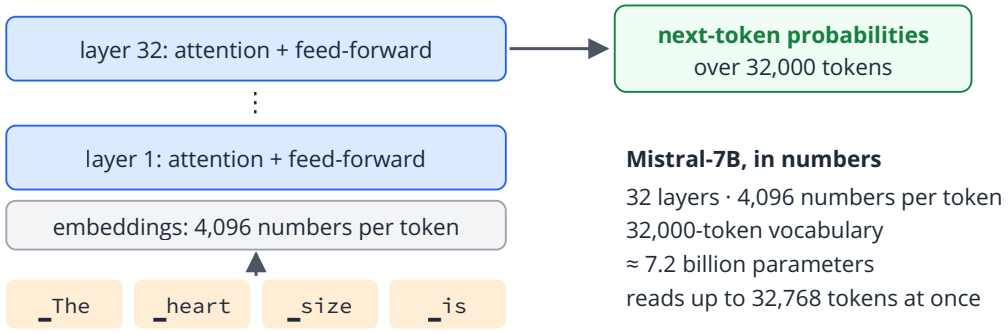
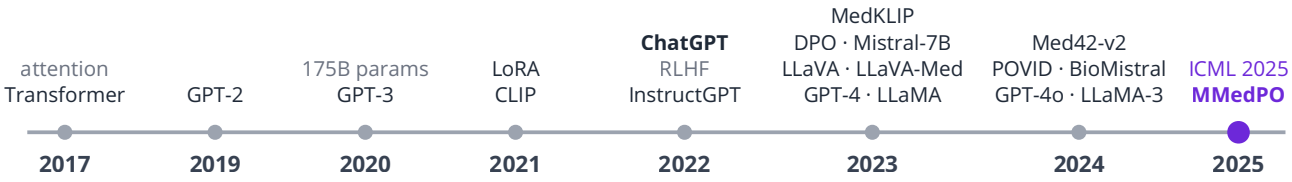


Figure 13. A transformer language model, simplified. The numbers on the right are read from the config of the LLaVA-Med v1.5 checkpoint on our cluster.

The five years you missed, on one line



Language: models grew from millions to hundreds of billions of parameters and became chat assistants.

Vision: CLIP linked images and text; LLaVA plugged an image encoder into an LLM; LLaVA-Med specialised it to medicine.

Alignment: RLHF, then the simpler DPO, taught models to prefer good answers over bad ones.

MMedPO combines all three strands for medical images.

Figure 14. A timeline of the work MMedPO builds on. Every name on it is explained somewhere in this guide.

From raw model to helpful assistant: three training stages

Modern assistants are trained in stages, and MMedPO is a recipe for the *last* stage. Knowing where it sits explains much of the paper's vocabulary.

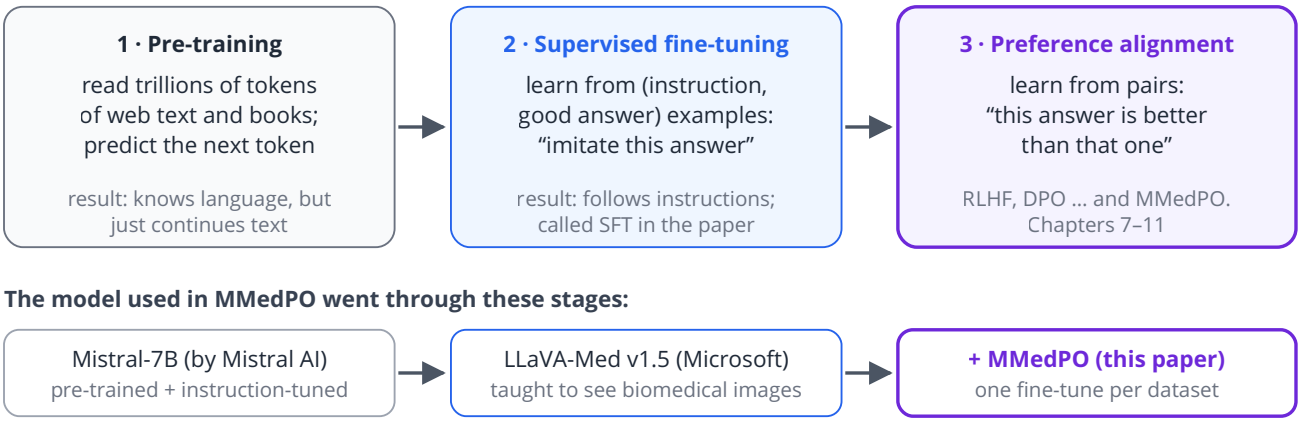


Figure 15. The three training stages (top) and the lineage of the model in the paper (bottom). In Table 1 of the paper, the "+ SFT" rows add an extra stage-2 step on each dataset before stage 3.

CHAPTER RECAP

- Text is a sequence of tokens. A language model outputs next-token probabilities and generates by looping.
- $\pi_\theta(y \mid x)$ is the probability the model gives to a whole answer. We work with its logarithm, a sum over tokens.
- Transformers use attention to decide which earlier tokens, including image tokens, matter.
- Training happens in stages: pre-training, supervised fine-tuning (SFT), then preference alignment. MMedPO is a stage-3 method.

CHAPTER 5

Teaching a language model to see

A language model only understands tokens. To answer questions about an X-ray, the image has to be turned into something that looks like tokens. This chapter shows how, using the exact design of the model in the paper.

An image is a grid of numbers

A digital image is a grid of **pixels**. In a greyscale X-ray each pixel is a number from 0 (black) to 255 (white). On the right is an 8×8 crop from the X-ray in Chapter 1, at the edge of a bright structure, with its actual pixel values.

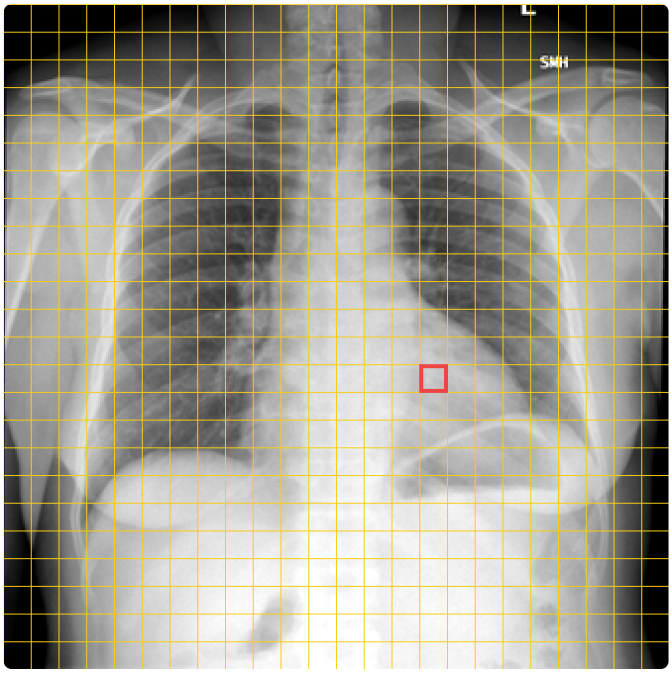


109	97	93	94	93	90	83	82
126	98	84	84	87	92	92	84
140	115	89	80	85	87	93	93
154	143	123	99	88	88	90	93
161	153	148	133	114	102	93	97
167	162	157	151	143	125	111	108
174	169	161	160	159	149	129	119
180	175	174	172	169	167	155	143

The numbers alone tell you very little; meaning comes from patterns across thousands of pixels. Learning to see those patterns is the job of a **vision encoder**.

Cutting the image into “visual words”: the vision transformer

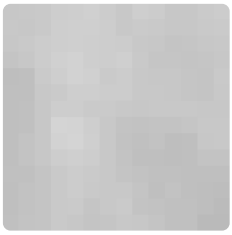
The vision encoder in the paper is a **vision transformer** (ViT). Its trick is to treat an image like a sentence: cut it into small square **patches**, turn each patch into a vector, and run a transformer over the patches just as over word tokens.



The model resizes every image to **336 × 336 pixels** and cuts it into patches of **14 × 14 pixels**. That gives 336 ÷ 14 = 24 patches across and 24 down:

24 × 24 = 576 patches

One patch (red square) enlarged:



Each patch becomes one vector, so the image enters the language model as **576 image tokens**. This number shows up in Guide 2: every prompt is about 600 tokens long, and 576 of them are the picture.

Figure 16. The X-ray from Chapter 1, resized to 336 × 336 and divided into the 24 × 24 grid of 14-pixel patches used by the model's vision encoder (CLIP ViT-L/14-336).

CLIP: teaching images and words to share a map

A vision encoder must produce vectors that *mean* something. The encoder in the paper comes from **CLIP** (OpenAI, 2021). CLIP was trained on 400 million images paired with their captions from the web. It has two encoders, one for images and one for text, and it learns by **contrast**: in a batch of images and captions, it pulls each image's vector towards its own caption's vector and pushes it away from the others.

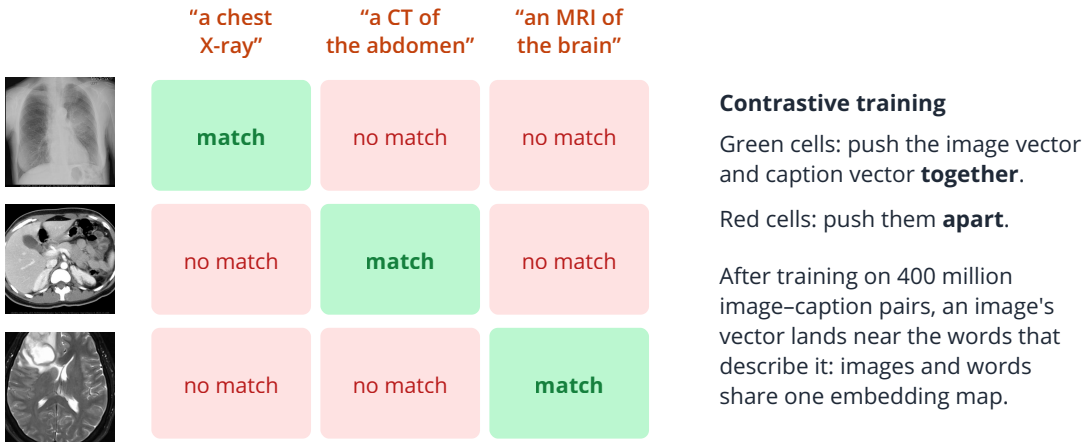


Figure 17. How CLIP learns (simplified, with our own images as stand-ins). This contrastive idea, "pull matching pairs together, push mismatched pairs apart", returns in a different form in DPO and in MedKLIP, the medical tool MMedPO uses to find lesions.

LLaVA: plugging an eye into a language model

LLaVA ("Large Language and Vision Assistant", 2023) is the design that the paper's model uses. The idea is simple: take a pre-trained vision encoder (CLIP) and a pre-trained language model, and connect them with a small **projector** network that translates CLIP's patch vectors into the language model's token space. The image patches then sit in the input sequence like extra words.

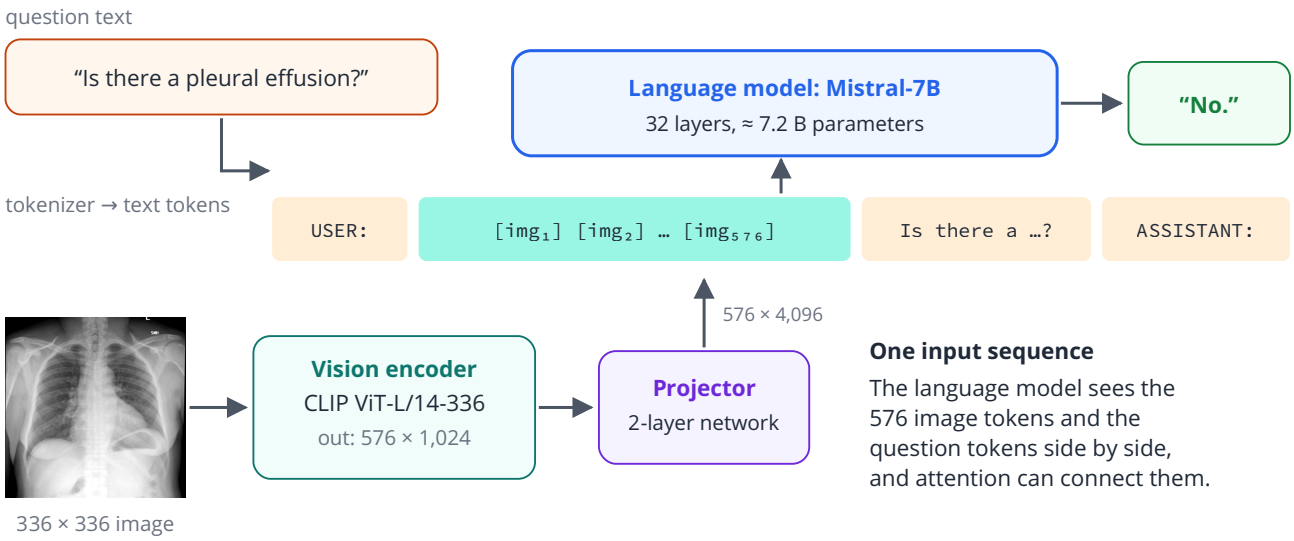


Figure 18. The LLaVA architecture used by LLaVA-Med v1.5, the base model of MMedPO. The image becomes 576 vectors via CLIP; the projector maps them to the language model's 4,096-number token format; they are inserted where the prompt says `<image>`. Everything after that is ordinary next-token prediction.

LLaVA is trained in two steps. First, only the projector is trained on image–caption pairs, so the language model learns to "read" CLIP's vectors. Then the projector and the language model are trained together on **visual instructions**: images with questions and good answers. This second step is supervised fine-tuning (stage 2 from Chapter 4) for images.

LLaVA-Med v1.5: the starting model in the paper

LLaVA-Med (Microsoft, 2023) applies the LLaVA recipe to biomedicine. It was trained on figure–caption pairs from PubMed Central (a free archive of medical research papers), plus question–answer conversations about those figures generated with GPT-4. Version 1.5 uses Mistral-7B as its language model. This is the model MMedPO fine-tunes; the paper calls it "LLaVA-Med-1.5 7B".

Part	What it is	Size
Vision encoder	CLIP ViT-L/14 at 336 px; uses its second-to-last layer	≈ 0.30 B params
Projector	two linear layers with a GELU bend between (`mlp2x_gelu`), 1,024 → 4,096	≈ 0.02 B params
Language model	Mistral-7B: 32 layers, 4,096 wide, 32,000-token vocabulary	≈ 7.24 B params
Prompt format	`USER: <image>\n question ASSISTANT:` (the "vicuna_v1" template)	576 image tokens
Total	stored in half precision (2 bytes per number)	**≈ 15 GB**

Table 3. LLaVA-Med v1.5 at a glance, read from the checkpoint we downloaded to the cluster.

CHAPTER RECAP

- Images are pixel grids; a vision transformer cuts them into 14-pixel patches: 576 for a 336-pixel image.
- CLIP learned to put images and matching words close together by contrastive training.
- LLaVA = CLIP + projector + language model; the image becomes 576 extra "tokens".
- LLaVA-Med v1.5 (Mistral-7B inside) is the model the paper starts from. In the paper's words it is a *Med-LVLM*: a medical large vision-language model.

CHAPTER 6

Hallucination: when the model stops looking

The paper's first sentence of motivation is: “Med-LVLMs encounter factuality issues due to modality misalignment, where the models prioritize textual knowledge over visual input, causing hallucinations.” This chapter unpacks that sentence.

What “hallucination” means

In AI, a **hallucination** is output that sounds fluent and confident but is not supported by the input, or is simply false. For medical images, three kinds are common:

- **Invented or missed findings**: reporting an effusion that isn't there, or “lungs are clear” when they aren't.
- **Wrong attributes**: the right finding on the wrong side, or the wrong size or severity.
- **Wrong knowledge**: naming the wrong anatomical structure or stating a false medical fact, like the “corpus callosum” answer in Chapter 2.

The paper uses **factuality** or **factual accuracy** for the opposite: how often the output agrees with the image and the ground truth.

Why it happens: two voices in the model

A vision-language model has two sources of information when it writes. One is the **image evidence** arriving through the 576 image tokens. The other is the **language prior**: everything the language model learned about what text usually looks like. Ideally the image wins whenever they disagree. In practice the language model is huge and was trained on vastly more text than images, so its habits often win.

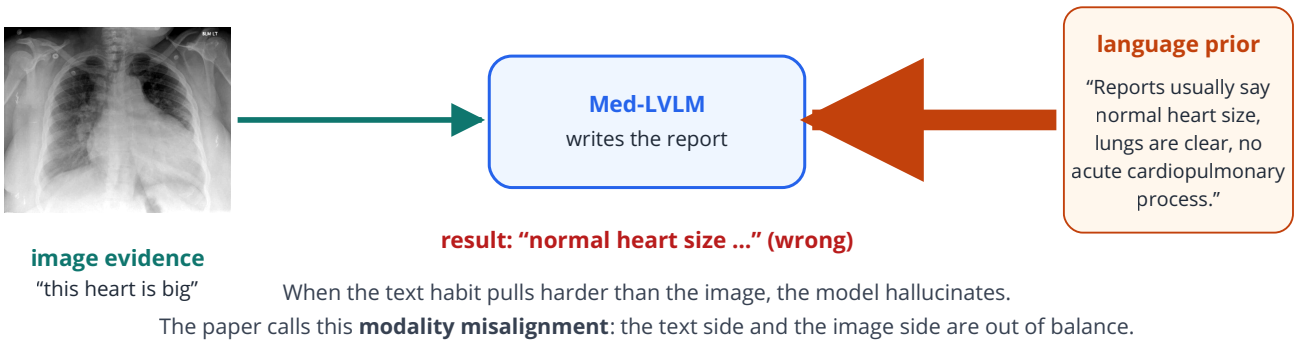


Figure 19. Modality misalignment. A “modality” is a type of input (image, text). Image from IU-Xray study CXR1965, whose true report says “moderate cardiomegaly”.

Evidence from our own cluster run

We ran the released MMedPO checkpoint for IU-Xray on all 590 test X-rays (Guide 2). The language prior is easy to see in the output:

- **561 of 590** reports (95%) contain the phrase “no acute cardiopulmonary …” and **563** contain “normal heart size”.
- The 590 reports contain only **48 distinct texts**. The model writes a handful of near-identical templates.
- A rough keyword search suggests that about **4 in 10** of the true reports mention at least one abnormality.

So even after MMedPO, the IU-Xray model leans heavily on the typical normal report. This is also a lesson about metrics (Chapter 12). Because most true reports are mostly normal sentences, a template can score reasonably on word-overlap metrics while missing the findings that matter.

> **WATCH OUT**
> This does not contradict the paper. The paper claims MMedPO *improves* factuality relative to other methods, and its numbers show it. It does not claim the problem is solved. Our observation is a reminder of how far there still is to go, and a good critical question to raise when you discuss the paper.

Why “just show it correct answers” is not enough

The obvious fix is more supervised fine-tuning: show the model images with correct reports. This helps, but it has a blind spot. SFT only ever says “make this answer more likely”. It never says “make *that* answer less likely”. A model can raise the probability of correct reports while keeping its favourite wrong template almost as probable, especially if the correct reports themselves are mostly normal sentences.

What is needed is training that shows the model a *bad* answer next to a *good* one and says: “prefer this one over that one.” That is **preference optimization**, the subject of Part III.

> **IN THE PAPER**
> Section 1 (Introduction) makes exactly this argument: earlier work applied preference optimization to medical models, but with pairs that were too easy and that ignored the image's lesion regions. MMedPO's contribution is to fix both. Part IV of this guide explains how.

CHAPTER RECAP

- Hallucination = fluent output not supported by the image. Factuality = the opposite.
- Cause: *modality misalignment*. The language prior (text habits) outweighs the image evidence.
- Our run shows it: 95% of the MMedPO IU-Xray reports use the same normal template.
- SFT only pushes good answers up; preference optimization also pushes bad answers down.

Part III

Learning from preferences

How to train a model with pairs of answers, one better and one worse, instead of single correct answers. This part covers the reward idea, RLHF, DPO (the method MMedPO modifies), and LoRA (the cheap way MMedPO applies it).

CHAPTER 7

Learning from comparisons: RLHF and DPO

Equation (1) in the paper is the DPO loss, and Equation (3), the MMedPO loss, is DPO with one extra factor. If you understand this chapter, you understand the mathematical core of the paper.

Preference data: a question, a winner and a loser

A **preference pair** is a triple: an input x, a **preferred** response y_w (w for "win") and a **dispreferred** response y_l (l for "lose"). The paper also calls them "chosen" and "rejected". A preference dataset D is just a list of N such triples, which is what the paper's notation $D = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^{N}$ says. The superscript (i) just numbers the examples.

Here is the example from the paper's Figure 1, retyped:

INPUT X
a chest X-ray
+ "Please generate a report based on this image."

PREFERRED Y_W (THE REAL REPORT)
The heart is mildly enlarged. Pulmonary vascularity is increased. There is again mild elevation of the right hemidiaphragm. Air space disease and/or atelectasis is noted in the right lung base.

DISPREFERRED Y_L
This image is a chest X-ray (CXR), captured in a frontal projection. The X-ray shows the presence of a **gallstone**. The gallstone is located in the right lobe of the **lung**.

The dispreferred answer is absurd: gallstones form in the gallbladder, not the lung. Hold on to that observation. Chapter 9 argues that this kind of pair is *too easy* to teach the model much.

Turning "better" into a number: rewards and the sigmoid

Suppose we had a function $r(x, y)$, a **reward**, that scores how good answer y is for input x. A classic model of human choices, the *Bradley–Terry model* (1952, originally for ranking chess players and sports teams), says the probability that people prefer y_w over y_l depends only on the difference in reward:

$$p(y_w \succ y_l) = \sigma(\, r(x, y_w) - r(x, y_l)\,)$$

The symbol $\succ$ reads "is preferred to". The **sigmoid** $\sigma(z) = 1 / (1 + e^{-z})$ squashes any number into a probability between 0 and 1: a reward gap of 0 gives 50/50, a large positive gap gives nearly 1. For example, rewards 2.0 and 0.5 give $\sigma(1.5) \approx 0.82$, an 82% chance the first answer is preferred. This is the formula in Section 2.2 of the paper.

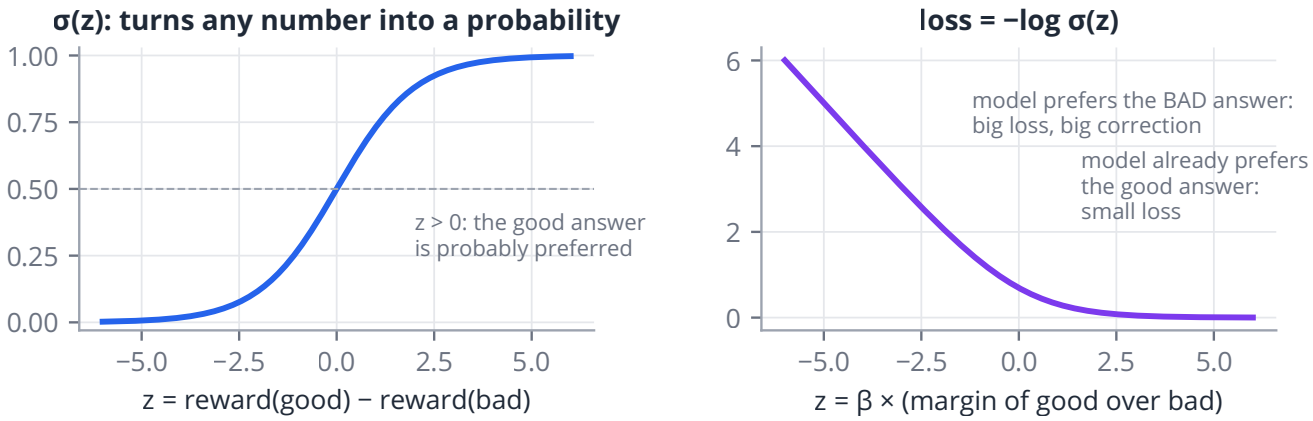


Figure 20. Left: the sigmoid turns a reward difference into a preference probability. Right: the training loss $-\log \sigma(z)$. It is large when the model favours the wrong answer ($z < 0$) and shrinks towards zero as the model favours the right one. DPO and MMedPO both minimise this curve.

The first solution: RLHF

The method that produced ChatGPT is **reinforcement learning from human feedback** (RLHF). It uses preference pairs in three steps:

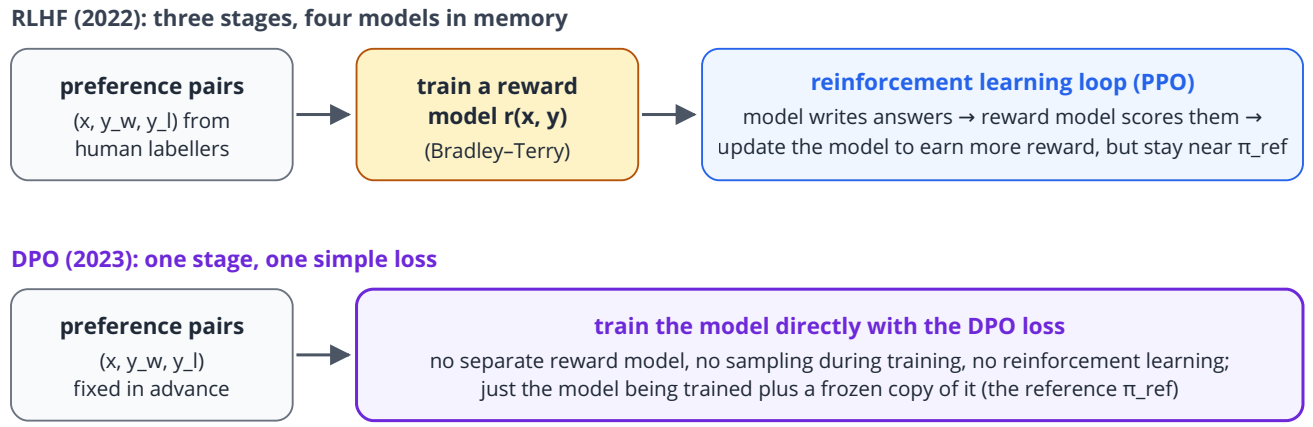


Figure 21. RLHF versus DPO. Both learn from the same kind of preference pairs. DPO reaches the same goal with a single loss function that looks like ordinary supervised training.

RLHF works, but it is complicated, expensive and notoriously unstable. In 2023 Rafailov and colleagues showed that the same objective can be optimised directly, with no reward model and no reinforcement learning. Their paper is titled *"Direct Preference Optimization: Your Language Model is Secretly a Reward Model"*, and DPO quickly became the default.

DPO, one symbol at a time

DPO's insight is that the model itself can act as the reward function. Define the reward of an answer as *how much more likely the model has become to say it, compared with where it started*:

$$r(x, y) = \beta \cdot \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}$$

Here π_θ is the model being trained and π_{ref} is a **frozen copy** of the model from before this training stage (the *reference model*). Plug this reward into the Bradley–Terry formula and turn it into a loss by taking –log. The result is the paper's Equation (1):

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,\, y_w,\, y_l) \sim D} \left[\log \sigma\!\left(\alpha \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \alpha \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right] \tag{1}$$

Reading it from the inside out:

Piece	Meaning
$\log \pi_\theta(y_w \mid x) / \pi_{\text{ref}}(y_w \mid x)$	How much more likely the model now makes the **good** answer than at the start. Positive = it went up. Computed as a difference of two log probabilities (Chapter 4).
$\log \pi_\theta(y_l \mid x) / \pi_{\text{ref}}(y_l \mid x)$	The same for the **bad** answer. We want this to go down.
their difference	The **margin**: how far the good answer has pulled ahead of the bad one.
α (usually written β)	A strength setting: how much a given margin counts. It also controls how far the model may drift from the reference. The authors' code uses 0.1.
$\log \sigma(\ldots)$ and the minus sign	Turns the margin into a loss via the right-hand curve in the sigmoid figure above: big when the margin is negative, near zero when it is large and positive.
$\mathbb{E}_{(x,\, y_w,\, y_l) \sim D}[\ldots]$	"Average over all triples in the dataset." $\mathbb{E}$ means expected value (an average); $\sim$ means "drawn from".

Table 4. The parts of the DPO loss. Training minimises this average with gradient descent, just as in Chapter 3.

> **WATCH OUT: NOTATION**
>
> Most papers, including the original DPO paper and the MMedPO code, call the strength setting β. The MMedPO paper calls it α in Equations (1) and (3), and then reuses α and β for something else in Section 3.3 (the clipping range of the weights). The paper also says "where π_θ represents the reference policy" under Equation (1); it means π_{ref}. Expect these slips; Chapter 14 has a full notation decoder.

A worked example

Take one preference pair. The log probabilities below are illustrative, but realistic in size for answers of a few dozen tokens. Use $\beta = 0.1$.

	$\log \pi_{ref}$ (start, frozen)	$\log \pi_\theta$ (after some training)	log-ratio
good answer y_w	−40.0	−38.0	**+2.0**
bad answer y_l	−35.0	−39.0	−4.0
margin = 2.0 − (−4.0) = 6.0 → z = 0.1 × 6.0 = 0.6 → $\sigma(0.6)$ = 0.646 → loss = −log 0.646			**0.437**

Table 5. One DPO step, in numbers. At the very start, $\pi_\theta = \pi_{ref}$, so both log-ratios are 0, the margin is 0, and the loss is −log 0.5 = 0.693.

Notice something important in the left column. At the start the model found the **bad answer more likely** (−35 beats −40). That is exactly the hallucination problem of Chapter 6: the wrong template sounds more natural to the model. DPO does not need the model to be right at the start. It measures each answer against its own starting point, so the gradient simply pushes the good answer up and the bad answer down until the margin is comfortably positive.

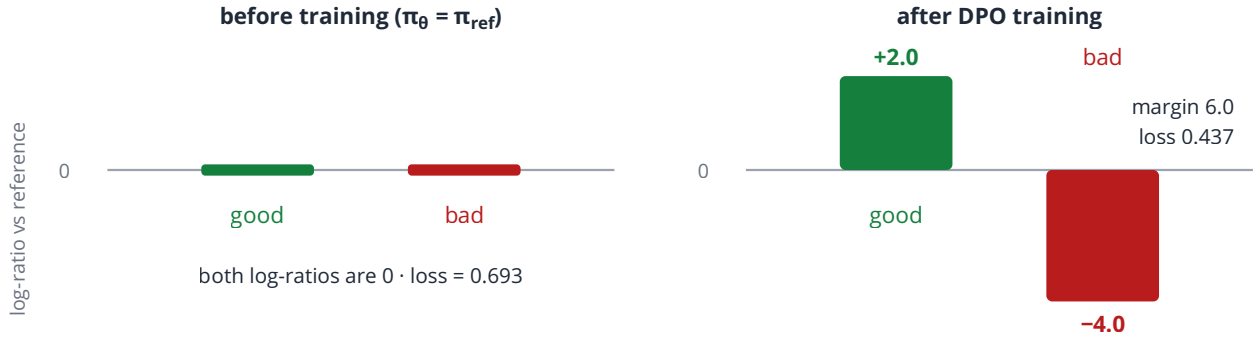


Figure 22. What DPO does to one pair. The bars show each answer's log-ratio against the frozen reference model. Training pushes the good answer's bar up and the bad answer's bar down.

> **KEY IDEA**
>
> DPO = "for every pair, increase how much more likely the good answer is than the bad one, relative to where the model started." Pairs the model currently gets wrong produce the biggest updates. Every pair counts *equally* in the average. That last point is what MMedPO changes.

Preference optimization for models that see

For vision-language models, the interesting question is **where the dispreferred answers come from**. Human labellers are too slow and expensive for thousands of medical images, so all the methods in the paper generate pairs automatically. The table summarises the baselines the paper compares against (its Section 4.1 and Appendix C):

Method	Where the pairs come from
DPO (2023)	Plain DPO on pairs of ground-truth answer versus the model's own answer.
Self-Rewarding (2024)	The model judges its own candidate answers ("LLM-as-a-judge") to build pairs, and repeats.
POVID (2024)	Two kinds of bad answer: GPT-4V inserts plausible hallucinations into correct answers, and the model answers a *noise-corrupted image*. MMedPO grows directly out of POVID: same research group, and its code reuses POVID's.
SIMA (2024)	The model generates answers and critiques them itself, using visual cues, to pick pairs.
FiSAO (2024)	Uses the model's own vision encoder to give word-by-word feedback, with no outside data.
STLLaVA-Med (2024)	Self-training for medical images; GPT-4o helps choose the preferred answers.

Table 6. The preference-optimization baselines in the paper. The first three were designed for language or general images; STLLaVA-Med is the only other medical one.

POVID's image trick is worth a picture in your head. If you blur or noise an image, the model can no longer see it properly, so its answer for the corrupted image is likely to be a hallucination driven by the language prior. Pairing "answer from the clean image" (good) against "answer from the corrupted image" (bad) teaches the model that answers *should depend on the image*. MMedPO's second idea (Chapter 10) sharpens this by corrupting only the lesion.

CHAPTER RECAP

- Preference data are triples (input, preferred, dispreferred).
- Bradley–Terry: preference probability = σ(reward gap).
- RLHF trains a reward model and then uses reinforcement learning. DPO skips both, using log-ratios against a frozen reference model as the reward.
- DPO loss = $-\log \sigma(\beta \times \text{margin})$, averaged over pairs with **equal weight**.
- Vision methods differ in how they make bad answers. POVID (hallucinated text plus noisy images) is MMedPO's direct ancestor.

CHAPTER 8

LoRA: fine-tuning a giant cheaply

The paper says “we apply LoRA fine-tuning” in one sentence. This chapter explains what that means, why it matters for cost, and why it adds nothing to inference time on our cluster.

The cost of changing 7 billion knobs

Ordinary fine-tuning updates every parameter. For a 7B model that means storing, for each of the 7.2 billion numbers, the number itself, its gradient, and two extra running averages that the standard optimiser (Adam) keeps. That is roughly 16 bytes per parameter, or over 100 GB of GPU memory, before even counting the images and text being processed. DPO also needs the frozen reference model in memory. And the result is a full 15 GB copy of the model for each dataset.

The LoRA idea: learn a small correction

LoRA (low-rank adaptation; Hu et al., 2021) freezes the original weights and learns only a small *correction* to them. Take one weight matrix W of the language model, say 4,096 × 4,096 numbers. Instead of changing W, LoRA adds the product of two thin matrices, B (4,096 × r) and A (r × 4,096), where the **rank** r is small:

$$W' = W + \frac{\alpha}{r} \cdot B\,A$$

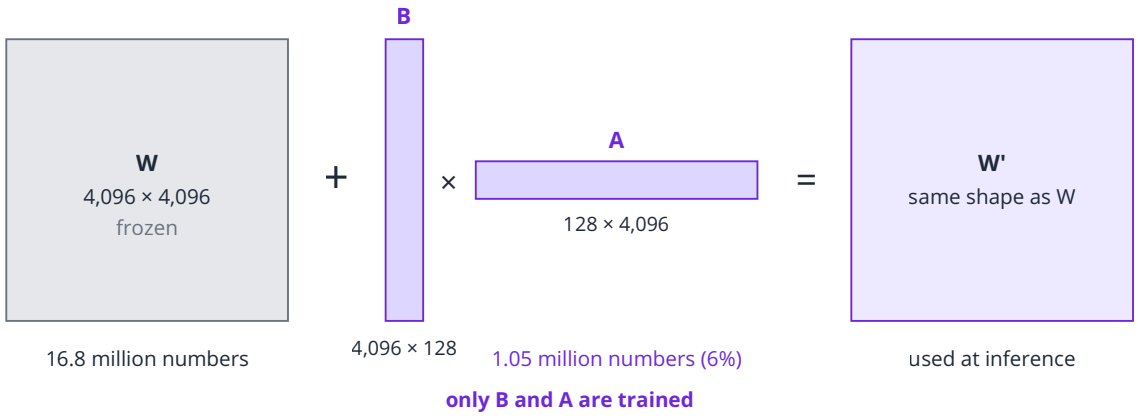


Figure 23. LoRA for one 4,096 × 4,096 weight matrix with rank r = 128, the setting used by MMedPO. The correction BA has the same shape as W but is described by 16 times fewer numbers.

Why should a small correction be enough? The base model already knows medicine and language. Fine-tuning only needs to *adjust* its behaviour (“look at the image more, prefer findings over templates”), and such adjustments turn out to be simple enough to describe with a low-rank product.

Setting in MMedPO's training script	Value	Meaning
rank r	128	width of the thin matrices
scale α (lora_alpha)	256	correction multiplied by α/r = 2
where	all 7 weight matrices in each of the 32 layers	q, k, v, o (attention) and gate, up, down (feed-forward)
trainable numbers	≈ 335 million	about 4.6% of the language model
also trained	the projector	at a higher learning rate (2×10^{-5})
saved checkpoint	≈ 0.65 GB per dataset	versus 15 GB for a full copy

Table 7. MMedPO's LoRA configuration (from the authors' training script and the released adapter_config.json). Yet another α: LoRA's scale has nothing to do with DPO's α or the clipping α.

Merging: why LoRA costs nothing at inference

After training, $W + (\alpha/r)\cdot BA$ can be computed once and stored as an ordinary matrix W'. The merged model has exactly the same shape and speed as the original LLaVA-Med. This is what our inference runs did: load the base model, load the 0.65 GB MMedPO adapter, merge, then answer questions. It is why the cluster timings in Guide 2 apply equally to "LLaVA-Med" and "LLaVA-Med + MMedPO": **MMedPO changes the model's answers, not its speed**.

CHAPTER RECAP

- Full fine-tuning of a 7B model needs over 100 GB of memory and a full model copy per task.
- LoRA freezes W and learns a thin correction BA. MMedPO uses r = 128, about 335 M trainable numbers.
- After merging, the model is the same size and speed as before. MMedPO adds zero inference cost.

Part IV

MMedPO itself

The paper's method, step by step: what is wrong with earlier medical preference data, how MMedPO builds harder pairs from text and from images, how it scores each pair's clinical importance, and how those scores enter the loss.

CHAPTER 9

The problem MMedPO attacks

By now you know the ingredients: a medical vision-language model that hallucinates (Part II) and DPO, a way to train it on good-versus-bad pairs (Part III). Earlier work had already combined them. The paper's argument is that they were combined carelessly, in two specific ways.

Weakness 1: the bad answers are too easy

Remember the gallstone example: “The X-ray shows the presence of a gallstone … located in the right lobe of the lung.” Any medical model can tell this is nonsense, so it already prefers the real report by a wide margin. Look back at the DPO loss curve: when the model already favours the good answer strongly, the loss is close to zero, and so is the training signal. **The pair teaches almost nothing.**

The useful pairs are **hard negatives**: dispreferred answers that are wrong but plausible, the kind of mistake the model would actually make. “Normal heart size” for a mildly enlarged heart. “Left pleural effusion” when it is on the right. These sit on the steep part of the loss curve, where the model is confused, and they produce strong, useful updates.

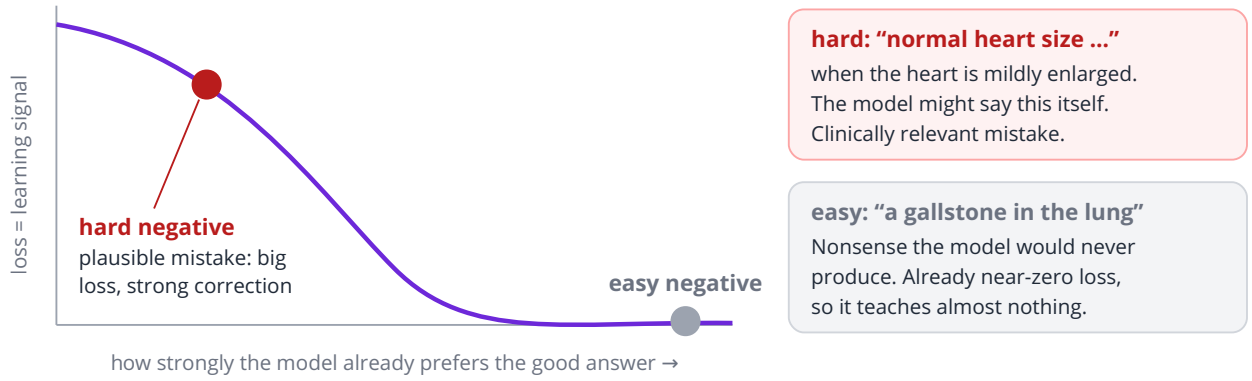


Figure 24. Why hard negatives matter (schematic). This is the paper's first observation: earlier medical preference data made pairs that were “easily distinguishable, reducing their effectiveness”. MMedPO responds in two ways. It makes the bad answers more plausible, and it *measures* how clinically meaningful each bad answer is so that the better pairs count more.

INTUITION

Training a radiology resident with multiple-choice questions whose wrong options are “gallstone in the lung” teaches nothing. The questions that teach are the ones whose wrong options are tempting: the wrong side, the wrong severity, a missed small effusion.

Weakness 2: the lesion is ignored

Earlier methods corrupted the *whole* image (POVID, Chapter 7) or did not use images at all to make negatives. But a radiologist's eye goes to the **lesion region**, the small part of the image where the abnormality is (Chapter 1). If you noise the whole image, the model can learn "blurry image means lower probability" without learning *where* to look. The paper wants dispreferred examples that arise specifically from **overlooking the lesion**.

The MMedPO plan in one picture

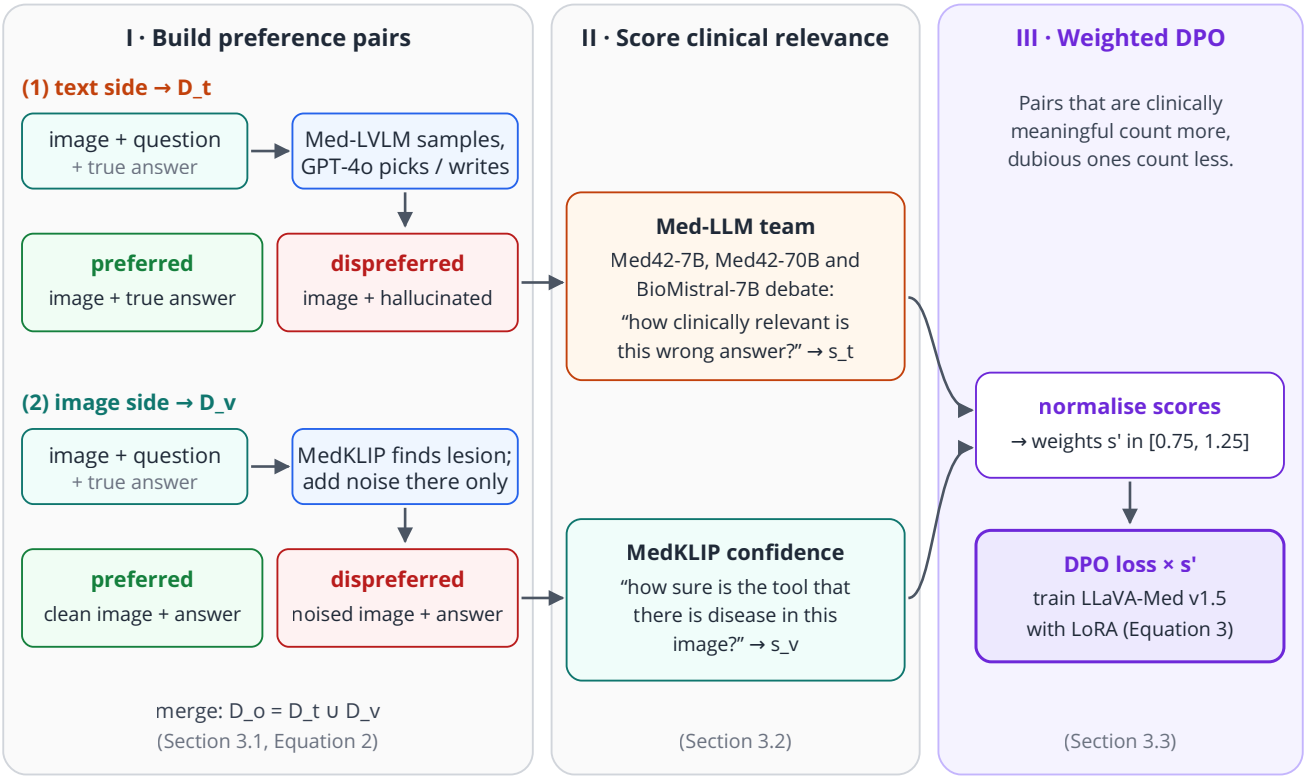


Figure 25. MMedPO in one picture: our redrawing of the paper's Figure 2. Columns I–III match Sections 3.1–3.3 of the paper and Chapters 10–11 of this guide.

> **IN THE PAPER**
>
> The name decodes as **M**ultimodal **Med**ical **P**reference **O**ptimization. "Multimodal" because the dispreferred side comes from both modalities: hallucinated *text* and noised *images*. "Clinical-aware" in the title refers to the weights.

CHAPTER RECAP

- Easy negatives give near-zero DPO loss and teach little; hard, plausible negatives teach a lot.
- Whole-image noise doesn't teach the model *where* to look; lesion-focused noise does.
- MMedPO = (I) build text-side and image-side pairs, (II) score each pair's clinical relevance, (III) DPO with those scores as weights.

CHAPTER 10

Step 1: building the preference data

Section 3.1 of the paper. Two strategies produce two sets of pairs, D_t from text and D_v from vision, which are then merged.

Strategy 1: plausible hallucinated answers (D_t)

For each training example (image x_v, question x_t, true answer y):

1. **Sample the target model several times.** Ask LLaVA-Med the question repeatedly with random sampling (Chapter 4) to get several candidate answers a. These are mistakes the model itself is prone to: natural hard negatives.
2. **Let GPT-4o judge.** GPT-4o (OpenAI's strongest multimodal model in 2024) compares each candidate with the true answer and picks the one with the most hallucination, meaning a clear conflict with the ground truth.
3. **Fallback: let GPT-4o write one.** If no candidate is wrong enough (the model got it right every time), GPT-4o rewrites the true answer with injected errors, using the prompt below.

THE PAPER'S PROMPT TO GPT-4O (ITS FIGURE 8)

"You are provided with a ground truth report: {gt report}. Please focus on the critical lesion descriptions within the report and inject hallucinations into these key descriptions to construct a dispreferred response. Ensure that the dispreferred responses contain significant medical inaccuracies. You may refer to the following examples for guidance: <example 1>, <example 2>. Your response should consist only of the hallucination-injected response."

The pair is: preferred = true answer, dispreferred = hallucinated answer, same image. An example of what such a pair looks like, using the report from Chapter 2:

PREFERRED (TRUE REPORT)

The heart is **mildly enlarged**. **Left** hemidiaphragm is elevated. There is no acute infiltrate or pleural effusion. The mediastinum is unremarkable.

DISPREFERRED (ILLUSTRATIVE INJECTION)

The heart is **normal in size**. **Right** hemidiaphragm is elevated. There is a **small left pleural effusion**. The mediastinum is unremarkable.

Notice how the injected errors target the "critical lesion descriptions", exactly as the prompt asks: the size of the heart, the side, an extra finding. The rest is copied. This makes the dispreferred answer fluent and plausible; only someone who reads the image can tell them apart.

Strategy 2: noise on the lesion (D_V)

The second strategy keeps the text the same and damages the *image*, but only where the lesion is.

Finding the lesion: MedKLIP

MedKLIP (Wu et al., ICCV 2023) is a chest X-ray model trained with knowledge extracted from radiology reports. Given an X-ray and a finding name, it outputs (a) a probability that the finding is present and (b) a **heatmap**: a map over the image showing where it thinks the finding is. MMedPO queries it with the generic word "disease" (paper Appendix B) and uses the heatmap $h = T(x_v)$ as the lesion region. (T stands for "tool".)

Adding the noise: Equation (2)

$$x^*_v = \sqrt{\xi_k} \cdot (x_v \odot h) + \sqrt{(1 - \xi_k)} \cdot (\epsilon \odot h) + x_v \odot (1 - h)$$

Symbol	Meaning
x_v, x^*_v	the original image and the noised image
h	the lesion mask: 1 inside the lesion region, 0 outside (from MedKLIP's heatmap)
$\odot$	pixel-by-pixel multiplication. $x_v \odot h$ keeps only the lesion pixels; $x_v \odot (1 - h)$ keeps everything else.
ϵ	random noise ("static"), a fresh random number for each pixel
k, ξ_k	the noise step (0 to 999) and how much of the original survives at that step. ξ_k is a product of numbers just below 1 ($\xi_k = \prod \xi_i$), so it shrinks from 1 towards 0 as k grows. In the authors' code: $\xi_{300} = 0.96$, $\xi_{600} = 0.54$, $\xi_{999} = 0.08$.

Table 8. Equation (2), decoded. Teal term: a faded copy of the lesion. Red term: noise, only inside the lesion. Grey term: the rest of the image, untouched.

This formula is borrowed from **diffusion models**, the image generators behind tools like DALL·E and Stable Diffusion. They are trained by gradually adding noise to images over many steps, and the $\sqrt{\xi}$, $\sqrt{(1 - \xi)}$ mixing is their standard noising step. You don't need to know anything else about diffusion to read this paper.

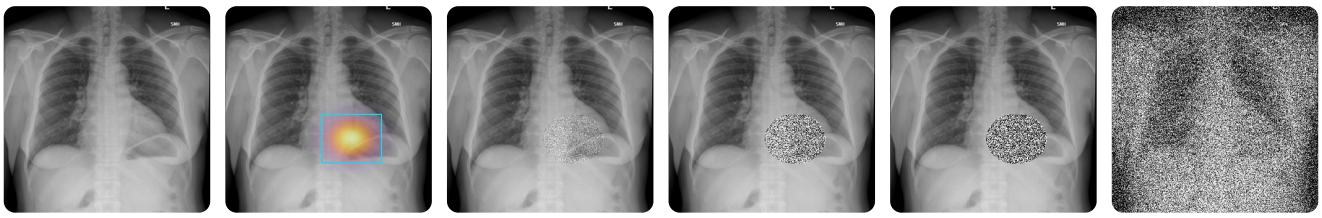


Figure 26. Equation (2) applied to a real IU-Xray image (study CXR3993, whose report says the heart is mildly enlarged and the left hemidiaphragm elevated), using the noise schedule from the authors' code. The lesion region here is *illustrative*: we placed it over the left heart border and hemidiaphragm by hand, because MedKLIP's weights are not on our cluster. Local noise wipes out exactly the evidence for the findings and leaves everything else intact. Global noise (right) damages everything; the paper compares the two in Table 4.

> **WATCH OUT: PAPER VERSUS CODE**
>
> The released curation script does something simpler than Equation (2). It replaces pixels whose heatmap value is above a threshold with pure random noise, which is equivalent to Equation (2) at the maximum step with a hard mask. Its training code also contains the diffusion-style noising function. The idea is the same either way: destroy the lesion and keep the rest. Small paper/code differences like this are common, and worth knowing about if you try to reproduce results.

Why “same answer, damaged lesion” teaches the model to look

This is the cleverest part of the paper, so take it slowly. In a D_v pair the **text is identical** on both sides; only the image differs:

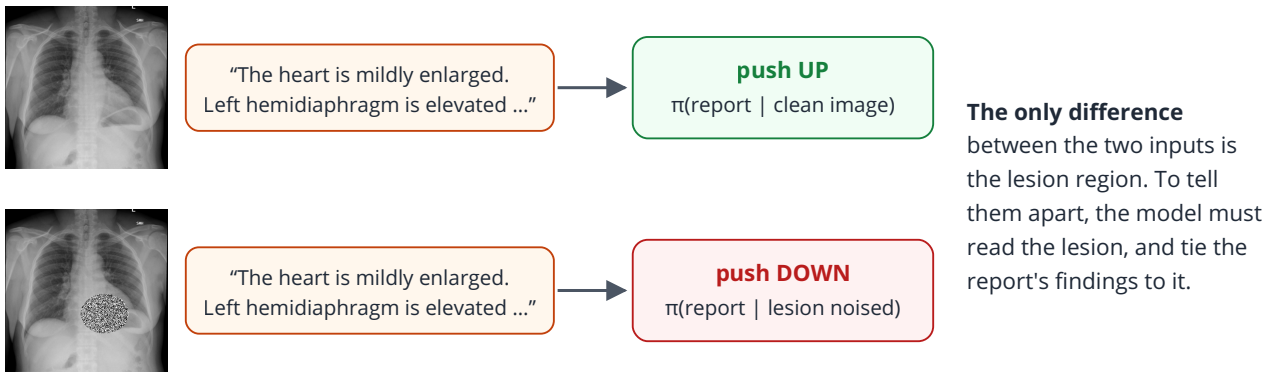


Figure 27. A D_v pair. The DPO margin can only grow if the model's confidence in “mildly enlarged” depends on actually seeing the heart border. With global noise, the model could learn the shortcut “noisy image means less confident” without looking at the lesion.

In other words, D_t teaches “given this image, prefer the correct findings over plausible wrong ones”, while D_v teaches “your belief in these findings should come from *this part* of the image”. Together they push against the language prior from both sides.

Merging the two sets

The paper merges both into one dataset $D_o = D_t \cup D_v$ whose elements have four parts: (x, x^*, y_w, y_l). The trick that lets one formula cover both:

- for a D_t pair, the "noisy" input x^* is just the original x (only the text differs);
- for a D_v pair, the dispreferred text y_l is just y_w (only the image differs).

Dataset	D_v (image pairs)	D_t (text pairs)	D_o total
IU-Xray (reports)	2,069	2,069	4,138
MIMIC-CXR (reports)	800	800	1,600
SLAKE (VQA)	4,919	4,919	9,838
VQA-RAD (VQA)	1,797	1,797	3,594

Table 9. Size of the preference training sets (paper Table 5). Each training example yields one pair of each kind. These are small datasets by modern standards, a few thousand pairs each.

CHAPTER RECAP

- D_t: the model's own sampled mistakes, picked by GPT-4o (or GPT-4o-injected errors) versus the true answer. Plausible, hard negatives.
- D_v: same true answer; the image's lesion region (found by MedKLIP) is replaced with noise via Equation (2).
- D_v forces the model's confidence to depend on the lesion region.
- Both merge into D_o with elements (x, x^*, y_w, y_l).

CHAPTER 11

Steps 2 and 3: scoring and weighted training

Sections 3.2 and 3.3 of the paper. Not every generated pair is equally useful: some injected errors are clinically trivial, and some MedKLIP heatmaps point at the wrong place. MMedPO scores each pair and lets that score scale its influence on training.

Scoring text pairs: a panel of medical language models

For a D_t pair, the question is: *how clinically meaningful is this wrong answer?* A meaningful mistake, like a wrong severity or a missed effusion, is a valuable hard negative. A meaningless one, like the gallstone in the lung, is not. Judging this requires medical knowledge but not the image, so the paper uses text-only **medical LLMs** (Med-LLMs): general LLMs further trained on medical text.

Because one judge can be biased, the paper uses a **panel** that debates until it agrees, an idea from earlier work on multi-agent debate (ChatEval):

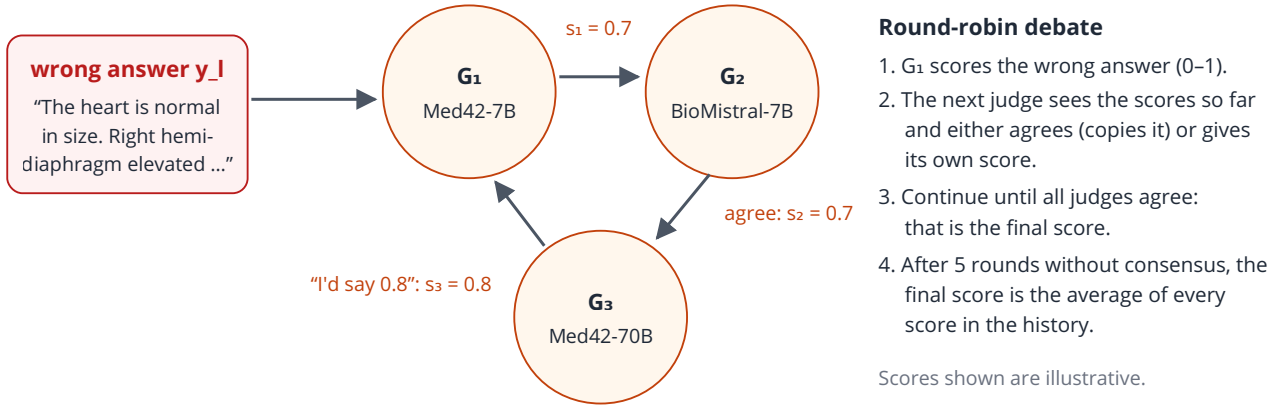


Figure 28. The multi-agent scoring of Section 3.2.1. The paper does not say in what order the judges speak; the order above is our choice. The score s is clinical relevance: higher means the wrong answer is a more meaningful, and so more useful, hard negative.

THE PAPER'S PROMPT TO EACH MED-LLM (ITS FIGURE 9)

"You are given a set consisting of a question, answer, and score. The question is: {prompt}, and the answer is: {rejected_value}. The provided score(s) for this answer is: {weighted_score}. These scores were assigned by other medical large language models based on evaluation criteria that assess the clinical value of the answer, emphasizing its relevance and specificity. The evaluation also considers whether the answer directly addresses the question without relying solely on general or common knowledge. Based on the question and answer, evaluate the score yourself. If you disagree with the provided score(s), assign a new score between 0.0 and 1.0 in your response. If you agree with the provided score(s), simply copy the score or respond with "agree." Response:"

The judges: **LLaMA3-Med42-7B** and **LLaMA3-Med42-70B** (Meta's LLaMA-3 further trained on medical data by M42 Health; 7 and 70 billion parameters) and **BioMistral-7B** (Mistral-7B further trained on PubMed Central articles). Mixing sizes and training sources is meant to cancel individual biases. Table 3 of the paper tests whether the panel beats a single judge (Chapter 13).

Scoring image pairs: the tool's own confidence

For a D_v pair the risk is different: if MedKLIP's heatmap points at the wrong place, the noise hits healthy tissue, and the pair teaches the wrong lesson. So the paper uses MedKLIP's **confidence** as the score. In the released code it is MedKLIP's probability that "disease" is present in the image. A confident detection means the noised region more likely contains the real lesion, so the pair gets more weight.

From scores to weights: normalisation

Raw scores are between 0 and 1, but feeding them in directly would shrink the whole loss (the average weight would be well below 1) and slow learning. The paper calls this "underfitting caused by an excessively small overall loss". So the scores are **normalised** into weights s' that average about 1, then **clipped** to a narrow range:

$$s' = \mathrm{clip}\left(1 + \frac{s - \mathrm{mean}(s)}{\mathrm{std}(s)} \cdot \sqrt{0.1}\ ,\ 0.75,\ 1.25\right)$$

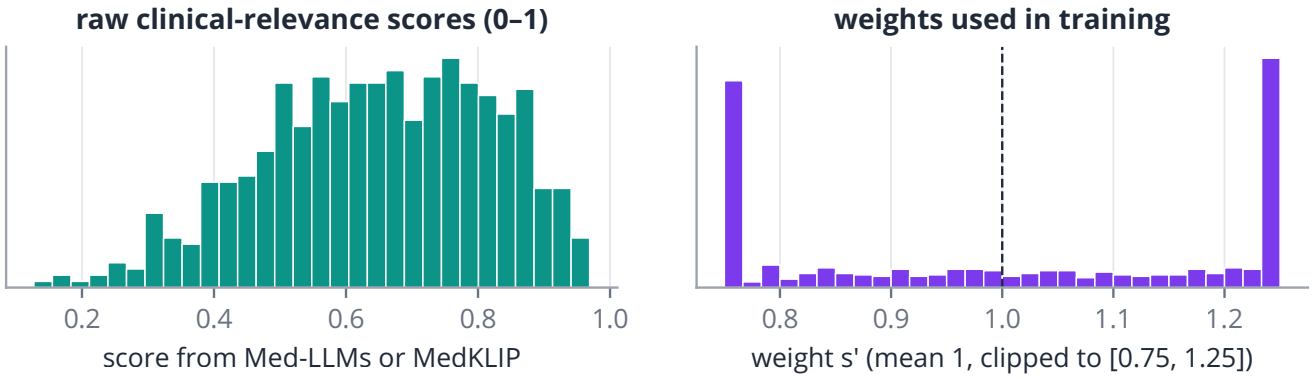


Figure 29. Normalisation (illustrative scores). Left: raw clinical-relevance scores. Right: the weights used in training, centred on 1 and clipped to [0.75, 1.25]. The spikes at the two ends are the clipped values. The most important pairs count at most 1.25 / 0.75 ≈ 1.7 times as much as the least important ones.

WATCH OUT: THE PRINTED FORMULA IS AMBIGUOUS

Section 3.3 writes $s' = (s - \mu)/\sigma$, "then we clip s' to values of $[\alpha, \beta]$", and Appendix B gives $\alpha = 0.75$, $\beta = 1.25$, $\mu = 1$, $\sigma^2 = 0.1$. Taken literally, a standardised score is centred on 0, so almost every value would be clipped. The reading above treats $\mu = 1$ and $\sigma^2 = 0.1$ as the *target* mean and variance, which matches the paper's words "mapping the scores to a fixed range while maintaining their mean and variance". The released code reads the weights precomputed from the data files, so it cannot settle the question. Either way the practical effect is the same: weights between 0.75 and 1.25 around an average near 1.

The MMedPO loss

Now everything comes together in Equation (3). Compare it with DPO's Equation (1); the differences are highlighted:

$$\mathcal{L}_{\text{mmedpo}} = -\mathbb{E}_{(x,\, x^*,\, y_w,\, y_l,\, s') \sim D_o} \left[s' \log \sigma\left(\alpha \log \frac{\pi_\theta(y_w \mid x)}{\pi_o(y_w \mid x)} - \alpha \log \frac{\pi_\theta(y_l \mid x^*)}{\pi_o(y_l \mid x^*)} \right) \right] \tag{3}$$

1. **The weight s'** multiplies each pair's DPO loss. Clinically important pairs pull harder on the parameters. In the authors' code this is literally one line: `weighted_losses = losses * batch_weight`.
2. **The dispreferred side uses x^***, the possibly-noised input. For text pairs $x^* = x$, so this is ordinary DPO. For image pairs it compares the same report under the clean and noised images.
3. π_o is just another name for the frozen reference model (π_{ref} in Equation 1).

A tiny numerical example: two pairs both have a DPO loss of 0.437 (as in the Chapter 7 example). Pair A has a clinically important negative ($s' = 1.25$); pair B's negative is trivial ($s' = 0.75$). Their contributions are 0.546 and 0.328. Since gradients scale with the loss, pair A moves the parameters about 1.7 times as hard as pair B.

KEY IDEA

MMedPO's training is DPO with two twists: the dispreferred side may be a *lesion-noised image* instead of a wrong text, and each pair's loss is multiplied by a *clinical-relevance weight* between 0.75 and 1.25. Everything else (the reference model, the sigmoid, gradient descent, LoRA) is standard.

Algorithm 1, annotated

The paper summarises the whole method as pseudo-code. Here it is with comments; M is the Med-LVLM, T the visual tool (MedKLIP), G the Med-LLM panel, N the noising step and Z the normalisation.

```
Input:  D = {(image x_v, question x_t, true answer y)}   # the training set
D_o = empty
for each (x_v, x_t, y) in D:
    # --- Step 1a + 2a: text-side pair ------------------------------------------
    a   = M(x_v, x_t)          # sample several answers from LLaVA-Med
    y_l = GPT(a, y)            # GPT-4o picks the most hallucinated one (or writes one)
    s_t = G(y_l)               # Med-LLM panel: clinical relevance of the wrong answer
    add (x_v, y, y_l, s_t) to D_o
    # --- Step 1b + 2b: image-side pair -----------------------------------------
    h    = T(x_v)              # MedKLIP heatmap: where is the lesion?
    s_v  = P(h | x_v)          # MedKLIP's confidence
    x*_v = N(x_v, h)           # Equation (2): noise only the lesion
    add (x_v, x*_v, y, s_v) to D_o
# --- Step 3: clinical-aware preference optimization -------------------------------
for each (x, x*, y_w, y_l, s) in D_o:     # in batches of 4, for 3 epochs
    s' = Z(s)                  # normalise and clip to [0.75, 1.25]
    update θ with Equation (3) # weighted DPO step on the LoRA parameters
```

The training recipe

Setting	Value
Starting model	LLaVA-Med v1.5 7B (Mistral-7B + CLIP ViT-L/14-336)
What is trained	LoRA adapters (r = 128, α = 256) on the language model, plus the projector
Optimisation	batch size 4, learning rate 1×10^{-7} (projector 2×10^{-5}), 3 epochs, cosine schedule
DPO strength (β, called α in the paper)	0.1 (code default)
Weight range	[0.75, 1.25]; Med-LLM debate up to 5 rounds; MedKLIP query “disease”
Hardware and time	4 × NVIDIA RTX A6000 (48 GB each); about 2–3 hours per dataset
Outputs	one model per dataset; four LoRA checkpoints released (IU-Xray, MIMIC-CXR, SLAKE, VQA-RAD). These are the ones we ran on the cluster.

Table 10. MMedPO training settings, from Section 4.1, Appendix B and the authors' training script. Data curation also needs the GPT-4o API, MedKLIP, and the three Med-LLMs (the 70B one alone needs about 140 GB of GPU memory at half precision).

CHAPTER RECAP

- Text pairs are scored by a debating panel of three Med-LLMs (0–1): how clinically meaningful is the wrong answer?
- Image pairs are scored by MedKLIP's confidence that disease is present.
- Scores are normalised to weights around 1, clipped to [0.75, 1.25].
- Equation (3) = DPO × weight, with the dispreferred side conditioned on the possibly-noised image x*.
- Training is cheap: LoRA, 3 epochs on a few thousand pairs, 2–3 hours on 4 GPUs.

Part V

The evidence

How the paper tests MMedPO (datasets, metrics, competing methods), what the numbers say, which parts of the method matter, and how to read the results with a critical eye.

CHAPTER 12

How the paper measures success

Section 4.1 of the paper lists datasets, metrics and baselines in a few lines each. This chapter expands each line so the results tables become readable.

The four test datasets

Dataset	Task	What it is	Test set used	Access
VQA-RAD (2018)	VQA	The first Med-VQA set written by clinicians: natural questions about X-ray, CT and MRI images from the MedPix teaching library.	451 questions 315 images	free
SLAKE (2021)	VQA	English–Chinese set of 642 images (CT, MRI, X-ray) and 14,028 questions, with organ and lesion annotations. The paper uses the English questions.	1,061 questions 641 images	free
IU-Xray (2016)	reports	Indiana University chest X-rays with their reports. Small and mostly normal studies.	590 reports	free
MIMIC-CXR (2019)	reports	Large chest X-ray archive from a Boston hospital (about 377,000 images). The paper tests on a 200-report subset.	200 reports	credentialed (PhysioNet)

Table 11. The evaluation datasets (paper Section 4.1, Appendix A). On our cluster we ran the first three in full; MIMIC-CXR needs a PhysioNet data-use agreement, so Guide 2 times the MIMIC checkpoint on IU-Xray images instead.

The metrics, with a worked example

A computer has to score thousands of answers automatically, so the paper uses simple word-matching metrics.

For VQA

- **Accuracy** (closed questions): the fraction of yes/no questions answered correctly.
- **Recall** (open questions): the fraction of the words in the correct answer that appear in the model's answer. If the truth is “left lung” and the model says “the abnormality is in the left lung”, recall is 2/2 = 100%. If the model says “right lung”, recall is 1/2 = 50%.

For reports

All three were invented for machine translation or summarisation and count overlapping words between the generated report (the *candidate*) and the radiologist's report (the *reference*).

- **BLEU** (2002): what fraction of the candidate's word sequences of length 1, 2, 3 and 4 (“n-grams”) also appear in the reference, with a penalty for being too short. The paper's BLEU column is the average of BLEU-1 to BLEU-4.

- **ROUGE-L** (2004): based on the longest sequence of words that appears in both texts in the same order (not necessarily adjacent).
- **METEOR** (2005): matches single words, also counting word stems and synonyms, weights recall above precision, and penalises matches that are scattered.

Here is a small example that shows how these metrics behave, and their main weakness:

```
Reference: the heart is normal in size
```

```
Candidate A: the heart size is normal (same meaning)
```

```
Candidate B: the heart is not normal in size (opposite meaning)
```

	word precision (BLEU-1 style)	2-word precision (BLEU-2 style)	ROUGE-L (F-score)	recall of reference words
A (correct)	5/5 = 1.00	2/4 = 0.50	0.73	5/6 = 0.83
B (wrong)	6/7 = 0.86	4/6 = 0.67	**0.92**	**6/6 = 1.00**

Table 12. Word-overlap scores for two candidate sentences (computed by hand, without BLEU's length penalty). The clinically wrong sentence scores higher on most metrics, because adding the word “not” barely changes the overlap.

> **WATCH OUT**
> BLEU, ROUGE-L and METEOR measure *wording overlap*, not *clinical correctness*. A report can score well by copying the typical normal phrasing (recall the 95% template rate in Chapter 6), and a negation can flip the diagnosis while barely changing the score. Radiology AI papers increasingly add clinical metrics that check findings directly (for example, labelling each report for 14 findings and comparing the labels). MMedPO reports only the word-overlap metrics. Keep this in mind when reading the report-generation gains.

Baselines and the “+ SFT” block

MMedPO is compared with the untuned LLaVA-Med v1.5 and six preference methods: DPO, Self-Rewarding, POVID, SIMA, FiSAO and STLLaVA-Med (described in Chapter 7). Every method is applied to the same starting model and data, so differences come from the method.

Table 1 of the paper has two blocks. In the top block each method starts from LLaVA-Med v1.5 directly. In the bottom block (“+ SFT”), the model is first fine-tuned on each dataset's correct answers (supervised fine-tuning, stage 2 from Chapter 4), and then each preference method is applied. The bottom block asks: does MMedPO still help a model that has already been tuned on the target data?

“Improved by 14.2%”: absolute versus relative

Papers report improvements in two ways, and it matters which:

- **Absolute** (percentage points): SLAKE closed accuracy goes from 62.02 (DPO) to 73.08 (MMedPO), a gain of 11.06 points.
- **Relative** (percent): 73.08 / 62.02 = 1.178, a 17.8% improvement.

The paper's headline numbers are averaged relative improvements. For example, “MMedPO outperforms DPO by 15.8% on open-ended questions” is the average of +9.5% (SLAKE: 53.99 vs 49.30) and +22.2% (VQA-RAD: 36.36 vs 29.76). We checked these and they reproduce exactly. Relative numbers look large when the starting score is small: going from 10 to 15 is “+50%”.

> **CHAPTER RECAP**
>
> - Two VQA datasets (VQA-RAD, SLAKE: open and closed questions) and two report datasets (IU-Xray, MIMIC-CXR).
> - VQA: accuracy (closed) and recall (open). Reports: BLEU, ROUGE-L and METEOR, all word-overlap metrics.
> - Word overlap ≠ clinical correctness: a negation can score higher than a correct paraphrase.
> - Six preference baselines, each with and without a prior SFT step. Headline gains are *relative* percentages.

CHAPTER 13

What the results show

Section 4.2–4.4 of the paper. First the main comparison, then the ablations that test each component, then what our own cluster run adds, and finally a critical reading.

Main results (Table 1)

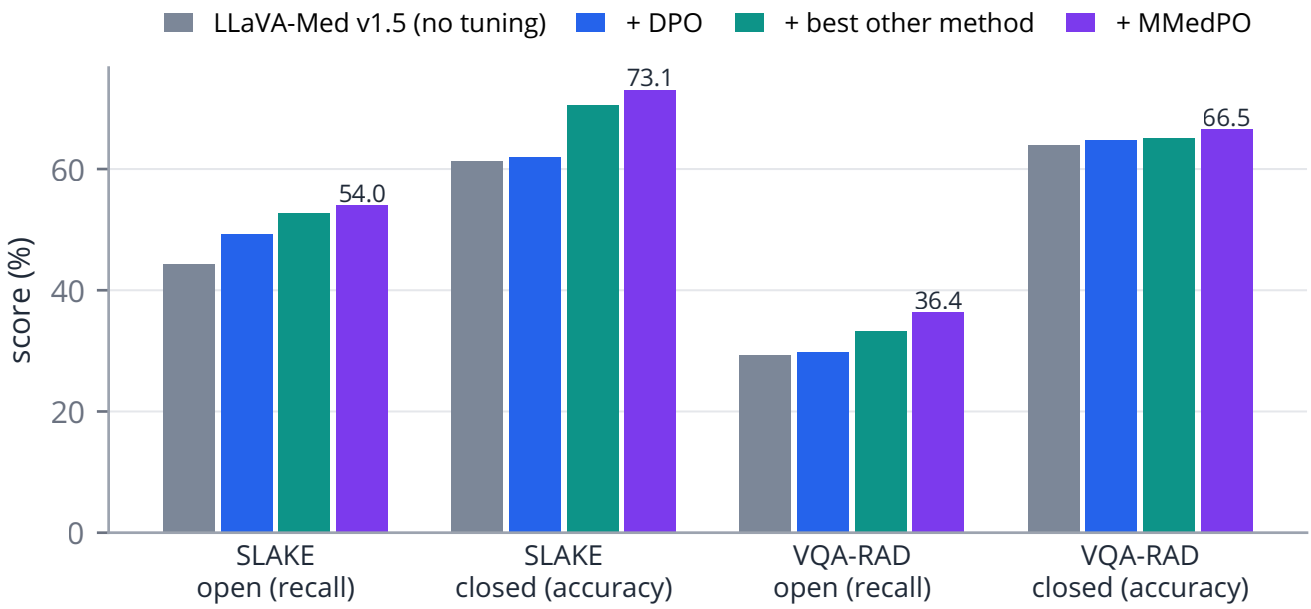


Figure 30. Medical VQA results (paper Table 1, top block). “Best other method” is the highest score among Self-Rewarding, POVID, SIMA, FiSAO and STLLaVA-Med for each column. MMedPO is best in every column.

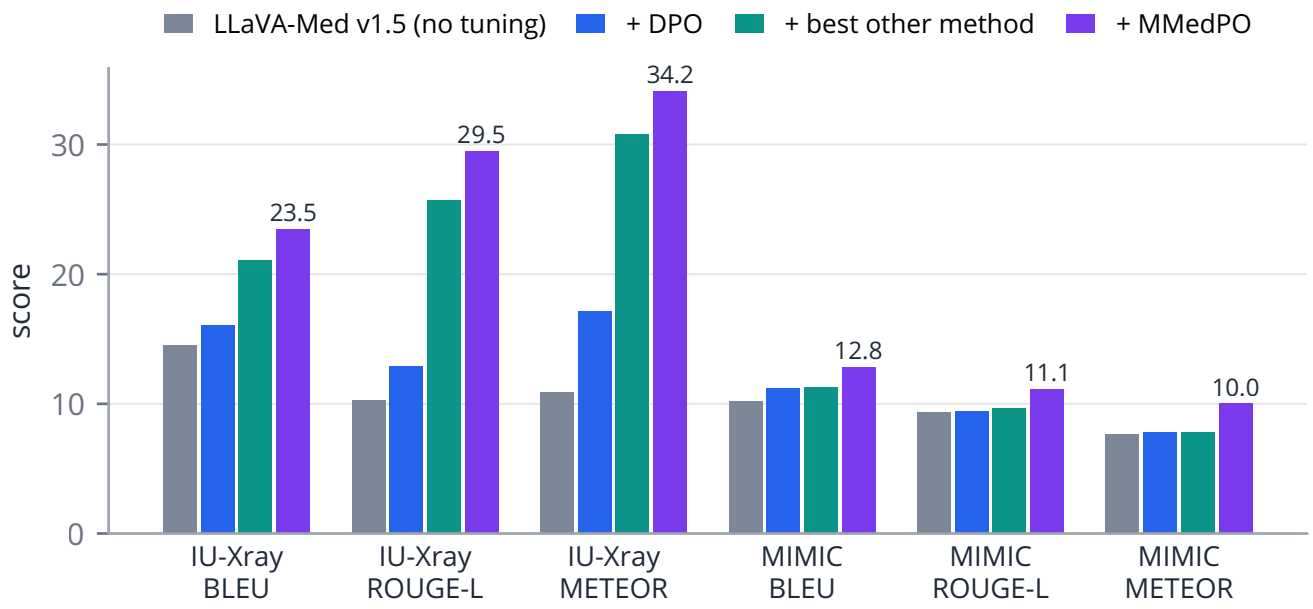


Figure 31. Report-generation results (same table). The jump on IU-Xray is large: METEOR goes from 10.95 to 34.16. On MIMIC-CXR all methods are close; MMedPO's lead is about 1–2 points.

Some observations worth making yourself when you read Table 1:

- **MMedPO wins every column**, in both blocks (with and without SFT).
- **Open-ended VQA gains more than closed**: +15.8% versus +10.3% relative to DPO. The paper reads this as MMedPO being "particularly effective in guiding accurate open-ended generation".
- **Methods that corrupt images do well on reports.** POVID, SIMA and FiSAO all jump on IU-Xray compared with plain DPO, and MMedPO, which also uses image-side negatives, jumps furthest. This fits the ablation below, which finds that report quality comes mainly from the image-noise pairs.
- **MIMIC-CXR barely moves for anyone.** It is a much more varied dataset than IU-Xray, and only 800 + 800 training pairs were used.

Model	SLAKE open	SLAKE closed	RAD open	RAD closed	IU BLEU	IU ROUGE-L	IU METEOR	MIMIC METEOR
LLaVA-Med v1.5	44.26	61.30	29.24	63.97	14.56	10.31	10.95	7.71
+ DPO	49.30	62.02	29.76	64.70	16.08	12.95	17.13	7.80
+ MMedPO	**53.99**	**73.08**	**36.36**	**66.54**	**23.49**	**29.52**	**34.16**	**10.03**
+ SFT	50.45	65.62	31.38	64.26	22.75	28.86	33.66	8.75
+ SFT + DPO	53.50	69.47	32.88	64.33	23.07	29.97	34.89	9.10
+ SFT + MMedPO	**55.23**	**75.24**	**34.03**	**67.64**	**24.00**	**30.13**	**35.17**	**10.20**

Table 13. A slice of the paper's Table 1. Note that SFT alone already reaches IU-Xray METEOR 33.66, almost as high as MMedPO's 34.16 without SFT. Most of the IU-Xray gain is simply "learn this dataset's writing style", which SFT also achieves.

Which parts matter? The ablations

An **ablation** removes or swaps one component and measures the change, to show that each piece earns its place. The paper has four.

Text pairs versus image pairs (Figure 3, Tables 8–9)

Preference data used	SLAKE open / closed	VQA-RAD open / closed	IU-Xray ROUGE-L	MIMIC ROUGE-L
Stage 1 only: hallucinated text (D_t)	49.39 / 65.87	32.42 / **69.85**	19.57	9.62
Stage 2 only: lesion noise (D_v)	51.25 / 68.51	31.09 / 62.87	29.02	9.85
Both (full MMedPO)	**53.99 / 73.08**	**36.36 / 66.54**	**29.52**	**11.13**

Table 14. Effect of the two data strategies (paper Tables 8 and 9, multi-LLM scoring). For reports, almost all the gain comes from the image-noise pairs. For VQA both help. One oddity: on VQA-RAD closed questions, text pairs alone (69.85) beat the full method (66.54).

Clinical-relevance weights on or off (Table 2)

	SLAKE	VQA-RAD	IU-Xray	MIMIC
Stage 1, no weights → with weights	55.65 → 57.62	47.23 → 48.67	10.95 → **15.66**	6.55 → 6.58
Stage 2, no weights → with weights	60.59 → 60.88	45.94 → 46.97	19.30 → **25.00**	7.17 → 7.24

Table 15. Turning the clinical-relevance score (CRS) weights on (paper Table 2; each number is the average of that dataset's metrics). The weights always help; the gain is small for VQA and MIMIC and large for IU-Xray.

One judge or a panel (Table 3) · Local or global noise (Table 4)

	SLAKE	VQA-RAD	IU-Xray	MIMIC
single Med-LLM → panel of three	56.09 → 57.53	48.67 → 51.14	15.67 → 15.86	6.58 → 6.86
global noise → local (lesion) noise	58.88 → 59.88	46.91 → 46.98	24.88 → 25.00	6.80 → 7.24

Table 16. The panel helps by a few percent, more on VQA. Local noise beats global noise on all four datasets, but by small margins (0.07 to 1 point).

A different base model (Figure 4)

The authors also apply MMedPO to **LLaVA-Med++**, a stronger model built on LLaMA-3 and trained on the much larger MedTrinity-25M dataset. MMedPO improves it on all four datasets, which suggests the method is not tied to one base model. The numbers appear only in the figure.

Looking inside: attention maps and cases

Section 4.4 shows qualitative evidence. **Figure 5** visualises how much attention the model pays to each image patch (Chapter 4). After MMedPO, attention concentrates more on lesion areas, which supports the claim that the model is "looking" more. **Figure 6** shows a report in which MMedPO correctly mentions the cardiac silhouette and a rib fracture, where the base model and DPO produce repetitive, clinically empty text. **Figure 7** shows a VQA case about a cyst. These are hand-picked examples. They illustrate the idea but are not statistical evidence.

What our cluster run adds

Guide 2 describes running the four released MMedPO checkpoints on our cluster. Besides timing, that run gives a few independent checks:

- ✓ **Accuracy reproduces.** With a simple yes/no matching rule, closed-question accuracy was **73.6**% on SLAKE (paper: 73.08%) and **64.9**% on VQA-RAD (paper: 66.54%). That is close, given our cruder scoring.
- ✓ **No extra cost.** The MMedPO model is LLaVA-Med with merged LoRA weights: same size (16.8 GB peak GPU memory) and same speed (about 0.1–0.2 s per question, 0.7–1.0 s per report on one B200).
- ✓ **The template problem remains.** 95% of the IU-Xray reports end with "no acute cardiopulmonary …", and abnormal cases such as the cardiomegaly study in Chapter 2 are still called normal.

Reading the results critically

A good paper reading ends with questions. These are the ones we would raise in a reading group:

1. **Metrics.** The report results use only word-overlap metrics (Chapter 12), with no clinical-finding metric and no radiologist review. The large IU-Xray gain is partly a style gain: SFT alone gets most of it.
2. **Small test sets, no error bars.** MIMIC-CXR is tested on 200 reports and VQA-RAD on 451 questions, with a single run each. Many ablation differences are below one point and may not be statistically meaningful.
3. **The weights are narrow.** Weights range only from 0.75 to 1.25. That a 1.7× maximum reweighting moves IU-Xray from 19.3 to 25.0 (Table 2) is striking, and worth asking about.
4. **Headline percentages.** The VQA percentages reproduce exactly against DPO. The report ones do not match "the best baseline" as stated. We could reproduce IU-Xray's 61.9% only as MMedPO's BLEU versus the *untuned* LLaVA-Med (23.57 vs 14.56, from Table 7). Tables 1 and 7 also disagree on some BLEU averages (base-model MIMIC: 10.25 vs 11.11). Trust the absolute numbers.

5. **MedKLIP outside chest X-rays.** MedKLIP was trained on chest X-rays, yet SLAKE and VQA-RAD include brain MRI and abdominal CT. The paper does not say how lesion regions are found for those images.
6. **Dependence on external models.** Curation needs the GPT-4o API (cost, reproducibility, and sending medical data to an outside service) plus a 70B Med-LLM.
7. **One model per dataset.** Each checkpoint is trained and tested on a single dataset; whether an IU-Xray model helps on MIMIC-CXR is not tested. Our run of the MIMIC checkpoint on IU-Xray images produced varied reports (437 distinct texts), unlike the IU-Xray checkpoint (48). Domain matters.

KEY IDEA

The method's two ideas, *lesion-local image negatives* and *clinical-relevance weighting*, are simple and supported by consistent (if sometimes small) gains. The evidence is strongest for image-noise pairs on report generation and weakest where the margins are under a point. The evaluation shares the field's usual weaknesses: overlap metrics and small test sets.

CHAPTER RECAP

- MMedPO beats all six baselines on all four datasets, with and without SFT.
- Report gains come mainly from lesion-noise pairs; VQA gains from both kinds of pair. The weights and the judge panel add smaller improvements.
- Our cluster run reproduces closed-question accuracy closely and confirms zero inference overhead.
- Caveats: overlap metrics, small tests, narrow weights, some inconsistent percentages, MedKLIP on non-chest images.

Part VI

Going further

A map from every section of the paper to this guide, a decoder for its notation, a reading list in a sensible order, questions to check yourself, and a glossary.

CHAPTER 14

Reading the paper and its neighbours

You now have everything the paper assumes. This chapter is a companion to keep open next to the PDF the first time you read it.

Section-by-section map

Paper section	What it does	In this guide
Abstract, Figure 1	The problem, the two kinds of dispreference, the weights, the headline gains	Ch. 7, 9
§1 Introduction	Hallucination from modality misalignment; prior preference data are too easy and ignore lesions	Ch. 6, 9
§2.1 Med-LVLMs	Image + query → next-token prediction	Ch. 4, 5
§2.2 Preference optimization, Eq. (1)	Bradley–Terry and the DPO loss	Ch. 7
§3 intro, Figure 2	The three-step framework	Ch. 9
§3.1, Eq. (2)	Hallucinated responses (D_t) and lesion noise (D_v)	Ch. 10
§3.2.1, §3.2.2	Multi-agent Med-LLM scores; MedKLIP confidence	Ch. 11
§3.3, Eq. (3), Algorithm 1	Normalisation and weighted DPO	Ch. 11
§4.1 Setup	Datasets, base model, LoRA, baselines, metrics	Ch. 8, 11, 12
§4.2, Table 1	Main comparison, with and without SFT	Ch. 13
§4.3, Tables 2–4, Figures 3–4	Ablations and the LLaVA-Med++ backbone	Ch. 13
§4.4, Figures 5–7	Attention maps, clinical-relevance example, case study	Ch. 13
§5 Related work	Factuality in Med-LVLMs; preference optimization	Ch. 7, reading list
App. A (Tables 5–6)	Training and test set sizes	Ch. 10, 12
App. B	Hyperparameters: “disease”, 5 rounds, $\alpha = 0.75$, $\beta = 1.25$, $\mu = 1$, $\sigma^2 = 0.1$, 4 × A6000	Ch. 11
App. C	Baseline descriptions	Ch. 7
App. D (Tables 7–9)	BLEU-1…4 detail; per-stage ablations	Ch. 13
App. E (Figures 8–9)	The GPT-4o and Med-LLM prompts	Ch. 10, 11
App. F (Figure 10)	More example cases	Ch. 13

Table 17. Where each part of the paper is explained. A good first read: Abstract → §1 → Figure 2 → §3 → Table 1 → §4.3, then the appendix as needed.

Notation decoder

Symbol	Meaning	Where
x_v, x_t, x	image (visual), query (text), and the pair $x = (x_v, x_t)$	§2.1
y, y_w, y_l	a response; preferred ("win"); dispreferred ("lose")	§2.2
$\pi_\theta(y \mid x)$	probability the model with parameters θ gives answer y to input x ("policy")	§2.2
π_{ref}, π_o	the frozen reference model. Same thing, two names (Eq. 1 vs Eq. 3). The text under Eq. (1) mistakenly writes π_θ.	Eq. 1, 3
$\sigma(\cdot)$	the sigmoid function	Eq. 1, 3
α in Eq. (1), (3)	DPO strength, called β in most other papers and in the code (0.1)	Eq. 1, 3
α, β in §3.3	lower and upper clip bounds of the weights (0.75, 1.25); *not* the α above	§3.3, App. B
μ, σ in §3.3	mean and spread of the scores (another use of σ!); App. B gives μ = 1, σ^2 = 0.1	§3.3
D, D_t, D_v, D_o	a preference dataset; text-side pairs; vision-side pairs; their union	§3.1
$M(\cdot)$, $T(\cdot)$, $G(\cdot)$	the Med-LVLM; the visual tool (MedKLIP); a Med-LLM judge	§3, Alg. 1
h, $\odot$, ϵ	lesion heatmap/mask; pixel-wise product; random noise	Eq. 2
k, ξ_k, $\bar{\xi}_k$	noise step; per-step keep factor; cumulative product (the text also writes $\bar{\xi}_t$)	Eq. 2
x^*_v, x_k, x^*	the lesion-noised image (the text uses all three names). In Eq. 3, $x^* = x$ for text pairs.	Eq. 2, 3
G_i, g, r, S, $\hat{s}$	i-th judge; number of judges; number of rounds; score history; average score when no consensus	§3.2.1
s, s_t, s_v, s'	clinical-relevance score; from judges; from MedKLIP; normalised weight	§3.2–3.3
$N(\cdot,\cdot)$, $Z(\cdot)$, $P(h \mid x_v)$	noising process; normalisation; tool confidence. (N also means "number of samples".)	Alg. 1
$\mathbb{E}_{\ldots\sim D}[\cdot]$	average over examples drawn from D	Eq. 1, 3

Table 18. Every symbol in the paper. The overloaded ones (α, β, σ, N, and the three names for the noisy image) cause most first-read confusion.

What to read next, in order

Each entry says why it is worth your time. The first group fills in the background at full depth; the second group are MMedPO's direct ancestors; the third is where the authors' line of work continues.

Foundations

1. **Vaswani et al., 2017. “Attention Is All You Need.”** The transformer. Read §3 for the architecture; skip the translation experiments. A gentler alternative is Jay Alammar's blog post “The Illustrated Transformer”.
2. **Radford et al., 2021. “Learning Transferable Visual Models From Natural Language Supervision” (CLIP).** The contrastive image–text training in Chapter 5; Figure 1 says it all.
3. **Liu et al., 2023. “Visual Instruction Tuning” (LLaVA)**, then **Liu et al., 2024. “Improved Baselines with Visual Instruction Tuning” (LLaVA-1.5).** The architecture of the paper's model.
4. **Li et al., 2023. “LLaVA-Med: Training a Large Language-and-Vision Assistant for Biomedicine in One Day.”** The starting model, and how its training data were built.
5. **Ouyang et al., 2022. “Training Language Models to Follow Instructions with Human Feedback” (InstructGPT).** RLHF, the method DPO replaces. Figure 2 is the classic three-step diagram.
6. **Rafailov et al., 2023. “Direct Preference Optimization: Your Language Model is Secretly a Reward Model.”** The most important one. §4 derives Equation (1); you now have all the pieces to follow it.
7. **Hu et al., 2021. “LoRA: Low-Rank Adaptation of Large Language Models.”** Short and readable; Figure 1 is our LoRA diagram.

Direct ancestors of MMedPO

1. **Zhou et al., 2024. “Aligning Modalities in Vision Large Language Models via Preference Fine-tuning” (POVID).** Same senior author (H. Yao). Hallucinated text plus noised images: MMedPO's blueprint.
2. **Wu et al., 2023. “MedKLIP: Medical Knowledge Enhanced Language-Image Pre-Training for X-ray Diagnosis.”** The lesion-finding tool.
3. **Chan et al., 2024. “ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate.”** The idea behind the Med-LLM panel.
4. **Sun et al., 2024. “STLLaVA-Med”** and **Hein et al., 2024. “Preference Fine-Tuning for Factuality in Chest X-Ray Interpretation Models Without Human Feedback.”** The other medical preference-tuning work MMedPO positions itself against.

Where the line of work continues

1. **Xia et al., 2024. “CARES: A Comprehensive Benchmark of Trustworthiness in Medical Vision Language Models.”** Measures the hallucination problem MMedPO targets.
2. **Xia et al., 2024. “RULE” (EMNLP 2024)** and **“MMed-RAG.”** The same group fights factual errors with *retrieval* (looking up similar cases) instead of preference training. A useful contrast.
3. Optional: **Ho et al., 2020. “Denoising Diffusion Probabilistic Models”**, if you want the full story behind Equation (2)'s noise schedule.

HOW TO READ AN ML PAPER QUICKLY

First pass (15 minutes): title, abstract, the overview figure, the main results table, the conclusion. Second pass (1 hour): the method section with a pen, rewriting each equation in your own words, as Chapters 7 and 11 did. Third pass, only if you will build on it: the appendix and the code. For MMedPO you have effectively done the first two passes already.

Check yourself

1. Why does a pair like "gallstone in the lung" contribute almost nothing to DPO training?

The model already strongly prefers the true report, so the margin is large and the loss $-\log \sigma(\beta \cdot \text{margin})$ is near zero, on the flat part of the curve. Near-zero loss means near-zero gradient.

2. In a D_v pair, what is different between the preferred and dispreferred sides?

Only the image: the dispreferred side has the lesion region replaced by noise. The text (the true answer) is identical.

3. What is π_{ref} for, and is it trained?

It is a frozen copy of the starting model. Log-ratios against it measure how much training has changed each answer's probability, and they keep the model from drifting too far. It is not trained.

4. Why don't the Med-LLM judges see the image?

They judge how clinically meaningful a *text* error is, which needs medical knowledge rather than the image. The paper argues that adding the image is unnecessary and may even hinder the judgement.

5. What does a weight $s' = 1.25$ mean in practice?

That pair's loss, and so its gradient, counts 25% more than an average pair. The least important pairs count 0.75.

6. Why is MMedPO's inference exactly as fast as LLaVA-Med's?

Training changed only LoRA adapters (and the projector), which are merged into the weights before use. The architecture and size are unchanged.

7. Which component drives the report-generation gains?

The lesion-noise image pairs (stage 2), helped by the clinical-relevance weights (Tables 2 and 8).

8. How many image tokens does the model see, and why?

576: a 336 × 336 image cut into 14 × 14 patches gives 24 × 24 = 576.

9. Why can a high BLEU score hide a clinically bad report?

BLEU counts overlapping words. A typical normal template overlaps heavily with mostly-normal references, and a negation barely changes the overlap.

10. What would you need to retrain MMedPO from scratch?

The datasets (MIMIC-CXR needs PhysioNet credentials), GPT-4o API access, MedKLIP weights, the three Med-LLMs (the 70B one needs about 140 GB of GPU memory), the authors' DeepSpeed training code, and roughly 4 GPUs for a few hours per dataset.

NEXT: GUIDE 2

Guide 2 walks through how we actually ran the released MMedPO models on the HGX cluster: what a cluster and Slurm are, how the software environment was built, what every script does, how the timings were measured, and how to ask the model your own questions.

Glossary

Ablation
An experiment that removes or swaps one component to measure its contribution.

Attention
The transformer mechanism by which each token draws information from the other tokens, image tokens included.

Autoregressive
Generating one token at a time, each conditioned on everything generated so far.

Batch, epoch, learning rate
Examples per update step; one full pass over the data; the size of each update step.

BLEU, ROUGE-L, METEOR
Word-overlap metrics between a generated text and a reference (Chapter 12).

Bradley–Terry model
Preference probability = sigmoid of the reward difference.

Cardiomegaly
An enlarged heart.

Checkpoint
A saved set of model parameters (or, for LoRA, of adapter parameters).

Closed / open-ended question
A fixed-answer (usually yes/no) question versus a free-text one.

CLIP
OpenAI's image–text model trained contrastively; its image encoder is LLaVA's "eye".

Clinical relevance score (CRS)
MMedPO's per-pair importance score, from the Med-LLM panel or MedKLIP's confidence.

Consolidation
Air spaces in the lung filled with fluid or pus, as in pneumonia.

CT
Computed tomography: cross-sectional slices built from many X-rays.

Diffusion model
An image generator trained by adding and removing noise in steps; the source of Equation (2)'s noise formula.

DPO
Direct Preference Optimization: a loss that trains directly on preference pairs, using log-ratios against a frozen reference model.

Embedding
The vector of numbers representing a token or image patch.

Factuality
Agreement of the output with the image and ground truth; the opposite of hallucination.

Fine-tuning; SFT
Further training of a pre-trained model; supervised fine-tuning imitates correct answers.

Finding
Anything a radiologist notes on an image.

fp16 / bf16
Half-precision number formats, 2 bytes per parameter.

GPT-4o
OpenAI's 2024 multimodal model; MMedPO uses it to pick or write hallucinated answers.

Gradient descent; backpropagation
Updating parameters in the downhill direction of the loss; the algorithm that computes that direction.

Ground truth (GT)
The correct answer or real report from the dataset.

Hallucination
Fluent output not supported by the input.

Hard negative
A dispreferred answer that is wrong but plausible, which makes it informative to train on.

Heatmap
A map over an image showing where a model's evidence for a finding lies.

Inference
Using a trained model to produce outputs, with parameters fixed.

Language prior
What the language model expects text to say, regardless of the image.

Lesion
Any area of abnormal tissue; the lesion region is where it appears in the image.

LLaVA, LLaVA-Med
Vision-language assistant = CLIP + projector + LLM; its biomedical version, the base model in MMedPO.

LLM
Large language model: a transformer trained to predict the next token on huge text corpora.

Log probability, log-ratio
The logarithm of a probability; the difference between two log probabilities.

LoRA
Low-rank adaptation: training a thin correction BA instead of the full weight matrix.

Loss
A number measuring how wrong the model is; training minimises it.

Med-LLM, Med-LVLM
A medical large language model (text only); a medical large vision-language model.

MedKLIP
A chest X-ray model that finds diseases and where they are; MMedPO's lesion tool.

Modality; modality misalignment
A type of input (image, text); the imbalance where text habits outweigh image evidence.

MRI
Magnetic resonance imaging: magnet- and radio-wave-based imaging, strong for soft tissue.

Parameters (weights), θ
The adjustable numbers of a model.

Patch
A small square of an image (14 × 14 pixels here) treated as one token.

Pleural effusion; pneumothorax
Fluid, or air, between the lung and the chest wall.

Policy (π)
Reinforcement-learning word for a model that produces outputs; here just “the model”.

Preference pair
(input, preferred answer, dispreferred answer).

Projector
The small network mapping CLIP's patch vectors into the LLM's token space.

Recall (VQA)
The fraction of ground-truth words that appear in the model's answer.

Reference model (π_{ref}, π_0)
A frozen copy of the starting model used in DPO.

Reward
A number scoring how good an answer is.

RLHF
Reinforcement learning from human feedback: reward model plus reinforcement learning.

Sigmoid (σ)
$\sigma(z) = 1/(1 + e^{-z})$, mapping any number to a probability between 0 and 1.

Temperature; sampling; greedy
How random generation is; picking tokens at random by probability; always picking the top token.

Token; vocabulary
A word or word piece; the fixed list of 32,000 tokens.

Transformer
The layered attention-based architecture behind all models in this guide.

Vision transformer (ViT)
A transformer over image patches.

VQA
Visual question answering: answering a question about an image.

X-ray
A shadow image; dense tissue appears white, air black.